Plataforma de estimación de producción de pozos shale de Vaca Muerta con Machine Learning en Google Cloud

Trabajo Final · Diplomatura en Inteligencia Artificial · Infraestructura para IA

Calvin Boyer (trabajo individual)

Noviembre de 2026

Índice

Caso de estudio y objetivo 4

Resultados principales 4

Respuesta a la devolución de la propuesta 4

Arquitectura 6

Diagrama 6

Componentes 6

Decisiones de diseño 7

Instrucciones de despliegue (reproducibles) 9

Requisitos 9

Fase 1. Datos y entrenamiento local 9

Fase 2. Kubernetes local (k3d) 9

Fase 3. GCP: preparación (una sola vez) 10

Fase 4. GKE 10

Fase 5. Pruebas de carga (GKE) 11

Apagar (costos) 11

Problemas frecuentes 11

Especificación de la API 13

Resumen 13

POST /predict 13

GET /wells/{idpozo} 15

GET /models 15

Métricas (/metrics) 16

Resultados de las pruebas y análisis 17

1. Arquitectura efectivamente usada 17

2. Ciclo de entrenamiento 17

3. Servicio de inferencia 21

Reflexión crítica 27

1. Alcances..... 27
2. Limitaciones del modelo..... 27
2b. Limitaciones de la plataforma (halladas en las pruebas)..... 27
3. Implicancias éticas..... 28
4. Qué falta para producción real..... 28
5. Continuidad (Curso 4, IA generativa)..... 29
Anexo: evidencias de la ejecución..... 30
1. Datos y entrenamiento local (PC, Windows)..... 30
2. Kubernetes local (k3d)..... 30
3. CI (equivalente al entregable 1.1 del lab)..... 32
4. GCP y Terraform (equivalente al 2.1)..... 33
5. GKE: entrenamiento, pods y /predict (equivalente al 3.1)..... 35
6. Grafana base, sin carga (equivalente al 4.1)..... 38
7. Carga y escalado (equivalente al 5.1 y 5.2)..... 39
8. Estado final (equivalente al 6.1)..... 49

Caso de estudio y objetivo

Denominación. Desarrollo y despliegue de una plataforma de estimación y consulta de producción mensual de pozos shale de Vaca Muerta mediante Machine Learning en Google Cloud.

Objetivo. Diseñar, desarrollar y desplegar un servicio de Machine Learning propio que estime la producción mensual de petróleo (`prod_pet`, m^3/mes) de pozos no convencionales de Vaca Muerta a partir de variables técnicas y operativas, con una API en Python sobre Kubernetes, entrenamiento y versionado del modelo, persistencia de resultados, infraestructura como código, CI/CD, observabilidad y autoescalado.

Datos. Datos abiertos de la Secretaría de Energía de la Nación (servicio ArcGIS REST): producción mensual por pozo (tabla 9), padrón de pozos (capa 0, para la edad del pozo) y trayectorias (capa 10, largo horizontal). Población: formación Vaca Muerta, recurso no convencional, subtipo shale, pozos petrolíferos en extracción efectiva, enero a junio de 2026: **12.321 registros de 2.212 pozos**, guardados como snapshot versionado (`csv.gz` + manifest con sha256).

Modelo. Regresión supervisada con un árbol de decisión (`DecisionTreeRegressor`, scikit-learn) dentro de un pipeline con imputación, escalado y codificación one-hot, en tres versiones con usos explícitos:

Versión	Variables	Uso
`v1`	TEF, profundidad, mes, sistema de extracción	Línea base de la propuesta (descriptivo)
`v2`	TEF, profundidad, edad del pozo, sistema de extracción, operadora, área	Descriptivo: requiere el TEF del mes, que se conoce al cerrarlo
`v2_notef`	`v2` sin TEF	Pronóstico: no usa información posterior al inicio del mes (modelo por defecto de la API)

Repositorio. https://github.com/calvinboyer-lang/tf-vaca-muerta-ml (código, charts, Terraform, workflows, documentación y capturas). Trabajo individual.

Resultados principales

- **Modelo:** en el holdout fijo de 432 pozos, `v2` alcanza MAE 483,5 m^3/mes y R^2 0,733; `v2_notef` (pronóstico) MAE 560,8 y R^2 0,635; `v1` MAE 696,1 y R^2 0,466. La edad del pozo explica la mayor parte de la mejora.
- **Reproducibilidad:** el mismo snapshot dio métricas idénticas en la PC, en k3d y en un Job dentro de GKE.
- **Servicio en GKE:** ≈ 60 req/s por réplica; 3 réplicas ≈ 180 req/s y 5 réplicas ≈ 250 req/s; con autoescalado (2–10) hasta 295 req/s; **0 % de errores** con la configuración final.
- **Hallazgos de operación:** sondas de salud que reiniciaban réplicas saturadas, requests de CPU que impedían escalar, conexiones persistentes que dejaban sin tráfico a las réplicas nuevas y el tope del node pool que deja réplicas en `Pending`. Cada uno está medido, explicado y, cuando correspondía, corregido.

Respuesta a la devolución de la propuesta

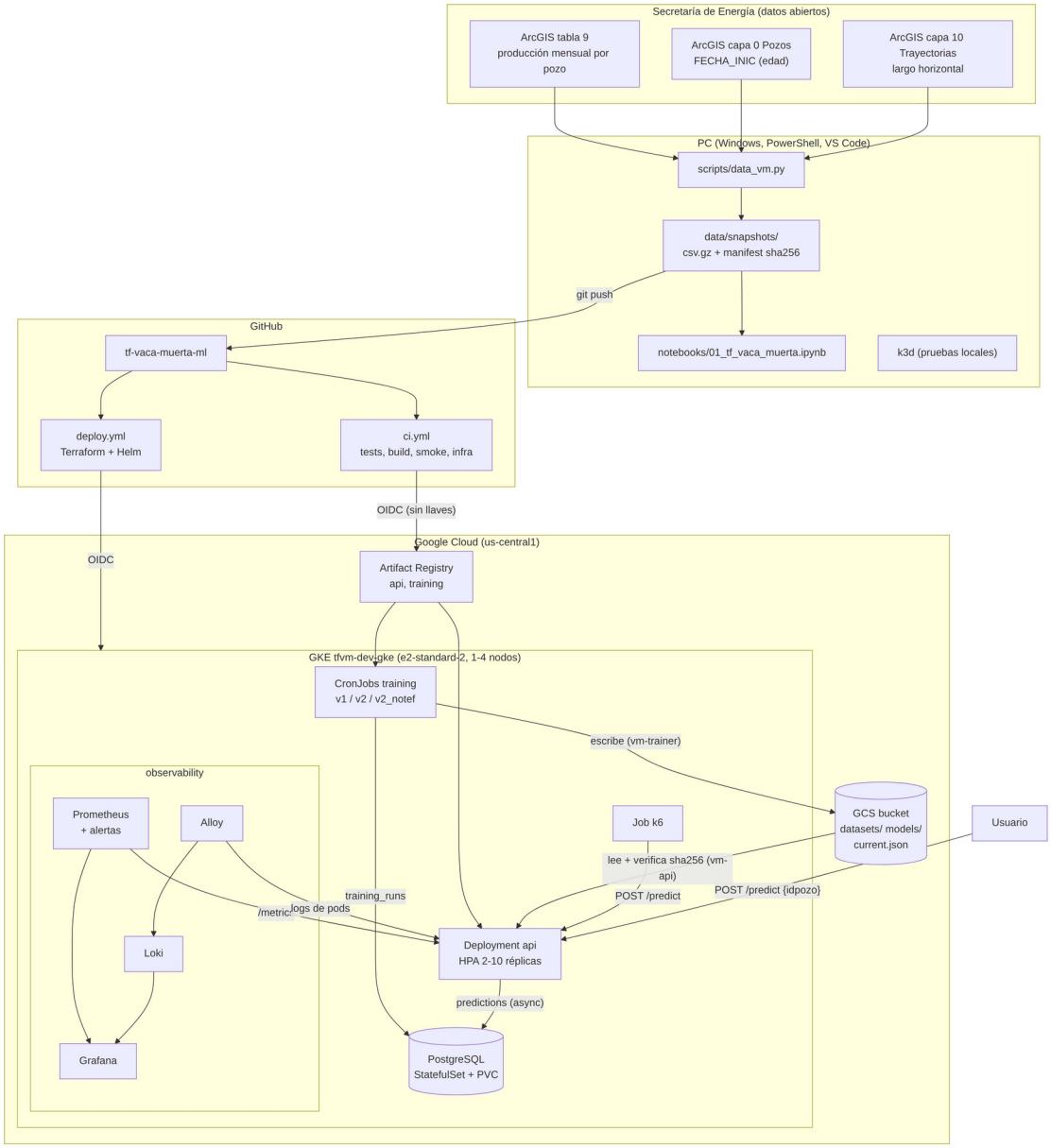
Punto de la devolución	Qué se hizo	Dónde
1. Reproducibilidad y R^2	Se reprodujo el R^2 original (0,476 con profundidad 6 y separación por pozo;	Resultados §2.2–2.4

Punto de la devolución	Qué se hizo	Dónde
	0,511 separando filas al azar). Hiperparámetros, separación y dataset quedan en el `metadata.json` de cada versión. Se informa promedio ± desvío sobre 10 separaciones y una validación temporal	
2. Mes del año vs. edad del pozo; TEF	Se agregó la edad del pozo (padrón oficial), la operadora y el área (`v2`), y una versión sin TEF para pronosticar (`v2_notef`). La API busca las variables del pozo a partir de su id	Resultados §2.3–2.4; Endpoints
3. Declinación	Se muestra y analiza: `v2` sigue la caída de los pozos jóvenes (−27 % contra −30 % real), pero en los maduros la estimación queda plana o escalonada	Resultados §2.7 y §3.0.1; Reflexión
4. Comparación y promoción	Holdout fijo por pozo (hash del id) y promoción solo si el MAE mejora ≥ 1 % y el intervalo bootstrap del 90 % excluye el cero. Se mostraron los tres casos (igual, mejora no significativa, mejora significativa)	Resultados §2.6
5. Definiciones de arquitectura	Carga del modelo desde Cloud Storage al arrancar y cada 2 min, con verificación de sha256; PostgreSQL como StatefulSet con PVC; 5 alertas; Alloy para los logs; Workload Identity con escritura solo para la cuenta de entrenamiento; trabajo individual	Arquitectura
Escalado	Se midió el techo por réplica y se justificó el umbral del HPA con esos números	Resultados §3.2–3.4

Las referencias del tipo «§3.2.1» remiten a la numeración interna de cada capítulo; las del tipo «captura 7.5b» o «evidencia 7.5b», al anexo de evidencias.

Arquitectura

Diagrama



Componentes

Componente	Implementación	Responsabilidad
Datos	`scripts/data_vm.py`, `vm_ml/data.py`	Descarga del ArcGIS, snapshot reproducible (sha256), edad del pozo y largo horizontal
Modelo	`vm_ml/features.py`	Pipeline scikit-learn (imputación, escalado, one-hot,

Componente	Implementación	Responsabilidad
		`DecisionTreeRegressor`) en un solo artefacto
Entrenamiento	`training/train.py` (imagen `training`, CronJob)	Valida, entrena, compara contra la vigente, versiona y registra
Registro de modelos	`vm_ml/registry.py` + bucket	`models/<m>/<versión>/{model.joblib,metadata.json}` y `current.json`
API de inferencia	`api/` (FastAPI, imagen `api`)	`/predict`, `/healthz`, `/metrics`, consultas de modelos y pozos
Base de datos	PostgreSQL (StatefulSet)	`training_runs`, `predictions`
Infraestructura	`terraform/`	Red, GKE, Artifact Registry, bucket, cuentas de servicio y Workload Identity
CI/CD	`.github/workflows/`	Tests, build, publicación y despliegue por OIDC
Observabilidad	`observability/`	Prometheus, Grafana, Loki/Alloy, 5 alertas, dashboard
Carga	`loadtest/`	k6 como Job dentro del cluster

Decisiones de diseño

1. El modelo no va dentro de la imagen. El template entrenaba Iris en el `docker build`. Acá el entrenamiento es un servicio aparte y la API descarga la versión vigente del bucket al arrancar. Así, entrenar y promover una versión no requiere reconstruir ni redesplegar la imagen, y cada versión queda trazable a su dataset (id + sha256), sus hiperparámetros y su separación.

2. Versiones inmutables + puntero. Cada entrenamiento escribe una carpeta nueva; solo `current.json` cambia. Volver atrás es reescribir el puntero. El bucket tiene versionado de objetos.

3. Comparación justa entre versiones (punto 4 de la devolución). El holdout se define por `sha256(salt:idpozo) % 100 < 20`: no depende de semillas ni del orden de las filas, es el mismo para todas las versiones y un pozo nuevo cae siempre del mismo lado. La versión vigente se **re-evalúa** sobre ese holdout con el dataset actual. Se promueve solo si el MAE mejora ≥ 1 % y el intervalo bootstrap al 90 % (remuestreo de pozos, pareado) de la mejora excluye el cero.

4. Tres versiones con usos distintos (punto 2). `v1` reproduce la propuesta; `v2` reemplaza el mes del año por la edad del pozo y suma operadora y área; `v2_notef` además quita el TEF, que solo se conoce al cerrar el mes, y es la única válida para pronosticar. La API sirve las tres y usa `v2_notef` por defecto.

5. La API completa las variables a partir del `idpozo`. Edad, operadora, área, profundidad y sistema de extracción salen del dataset del modelo; el usuario puede sobrescribir cualquiera. Si falta el TEF (v1/v2) se supone el mes completo y la respuesta lo informa en `fuentes`.

6. Carga y recarga del modelo (punto 5). - Al arrancar, la API lee `current.json`, descarga `model.joblib`, **verifica el sha256** y recién entonces `/healthz` responde 200. Si falla, el pod queda *not ready*: Kubernetes no le manda tráfico y el error aparece en `/models`, en los logs y en `model_reload_total{result="error"}`. - Cada réplica revisa `current.json` cada 120 s y cambia de versión de forma atómica. `POST /admin/reload` fuerza la revisión en la réplica que atiende. Si una carga falla, se conserva la versión anterior.

7. Seguridad del artefacto (`joblib` ejecuta código al cargar). - Escritura en el bucket: solo `vm-trainer` (`roles/storage.objectAdmin`). - La API (`vm-api`) solo tiene `roles/storage.objectViewer`. - Ambas acceden

por Workload Identity, ligadas a un ServiceAccount de Kubernetes concreto. No hay llaves JSON. - La API rechaza un artefacto cuyo sha256 no coincide con `current.json`. - Bucket con *uniform bucket-level access* y *public access prevention*.

8. PostgreSQL como StatefulSet y no Cloud SQL. Es un registro auxiliar de bajo volumen. Un StatefulSet con PVC cuesta solo el disco y funciona igual en k3d y en GKE, mientras que Cloud SQL suma una instancia que se paga por hora. La API escribe por lotes desde un hilo: si la base cae, la predicción no se frena (se cuenta en `db_write_errors_total`).

9. Un proceso por pod y escalado con réplicas. Así el uso de CPU del pod refleja la carga y el HPA (CPU 60 % de la *request*) decide bien. Los valores de *requests* y del umbral se justifican con las mediciones (Resultados).

10. Nodos e2-standard-2 con autoescalado 1–4. Con e2-medium (4 GB) Grafana se reiniciaba en el laboratorio. Diez réplicas de la API (384 Mi de *request* cada una), PostgreSQL y el stack de observabilidad necesitan más memoria. El tope de 4 nodos acota el costo de las pruebas.

11. Estado de Terraform separado del laboratorio (prefijo `tf-vaca-muerta/dev`, red `tfvm-dev-vpc`, cluster `tfvm-dev-gke`). Ambos entornos conviven en el mismo proyecto sin pisarse.

12. OIDC con un provider propio. El provider del laboratorio solo acepta tokens del repo del laboratorio (`attribute-condition`). Se crea otro en el mismo pool para este repo, sin modificar el del lab.

13. Observabilidad dentro del cluster. El stack del template, con más memoria. Las alertas son `PrometheusRule` evaluadas por Prometheus y visibles en Grafana. Alertmanager queda apagado, porque no hay a quién notificar.

Instrucciones de despliegue (reproducibles)

Windows + PowerShell en VS Code. Desde la raíz del repo. Los comandos se corren **de a uno**, revisando la salida.

Requisitos

Herramienta	Para qué	Verificación
Python 3.12	datos, entrenamiento, notebook	`python --version`
Docker Desktop	imágenes y k3d	`docker version`
k3d, kubectl, helm	Kubernetes local	`k3d version`, `kubectl version --client`, `helm version`
gcloud	GCP (bootstrap, bucket, credenciales)	`gcloud --version`
git	repo	`git --version`

Si PowerShell bloquea los scripts: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

Fase 1. Datos y entrenamiento local

```
git clone https://github.com/calvinboyer-lang/tf-vaca-muerta-ml.git
cd tf-vaca-muerta-ml
.\tasks.ps1 setup          # .venv + dependencias + kernel "TF Vaca Muerta (.venv)"
.\tasks.ps1 test           # tests con datos sintéticos
.\tasks.ps1 data-arcgis    # producción (tabla 9) + padrón (capa 0) + trayectorias (capa 10) -> data\raw
.\tasks.ps1 data-build     # data\snapshots\vm_shale_AAAAMM_AAAAMM.csv.gz + .manifest.json
.\tasks.ps1 train-all      # v1, v2, v2_notef -> .\artifacts
.\tasks.ps1 versions -Model v2_notef
```

Después se abre `notebooks/01_tf_vaca_muerta.ipynb` con el kernel **TF Vaca Muerta (.venv)** y se ejecuta todo.

El snapshot se versiona en git:

```
git add data/snapshots
git commit -m "datos: snapshot vm_shale_AAAAMM_AAAAMM"
git push
```

Fase 2. Kubernetes local (k3d)

Requiere haber corrido `train-all`, porque los pods leen `.\artifacts` montado como `/artifacts`.

```
.\tasks.ps1 k3d-up         # cluster "tfvm" (1 server + 1 agent)
.\tasks.ps1 k3d-images     # docker build api/training + k3d image import
.\tasks.ps1 k3d-deploy     # secret postgres-auth + postgres + training + api
kubectl get pods,hpa
.\tasks.ps1 port-forward   # en otra terminal: http://localhost:8081/docs
```

Prueba rápida:

```
Invoke-RestMethod http://localhost:8081/healthz
$id = (Invoke-RestMethod "http://localhost:8081/wells/sample?n=1").idpozo[0]
Invoke-RestMethod -Method Post http://localhost:8081/predict -ContentType
```

```
"application/json" -Body "{`"idpozo`": $id}"
Invoke-RestMethod http://localhost:8081/stats
```

Entrenamiento como Job dentro del cluster, para registrar el run en PostgreSQL:

```
.\tasks.ps1 k3d-train -Model v2
```

Prueba de carga local corta (k6 dentro del cluster):

```
.\tasks.ps1 loadtest -Vus 10 -Duration 1m -Label k3d-vus10
```

Al terminar: `.\tasks.ps1 k3d-down`.

Fase 3. GCP: preparación (una sola vez)

1. Abrí `scripts/bootstrap-gcp.ps1`, revisá las variables del bloque 0 (proyecto, pool) y corré los bloques uno por uno. Crea un **provider OIDC nuevo** solo para este repo y agrega roles a `ci-deployer`.
2. En GitHub, en **Settings → Secrets and variables → Actions → Variables**, cargá los valores que imprime el bloque 6: `GCP_PROJECT_ID`, `GCP_WORKLOAD_IDENTITY_PROVIDER`, `GCP_SERVICE_ACCOUNT`, `GCP_DEPLOY_SERVICE_ACCOUNT` y `GCP_REGION`.
3. Hacé un push (o re-ejecutá `ci`). El job `publish` sube `api` y `training` a Artifact Registry con tag `<sha>`.

   En el **primer** deploy el repositorio `tf-vaca-muerta` de Artifact Registry todavía no existe, porque lo crea Terraform. Orden: 1. `deploy` con `terraform=true` y `charts` vacío. 2. Re-ejecutar `ci` (publica las imágenes). 3. `deploy` de nuevo con los charts.

Fase 4. GKE

1. **Infraestructura:** Actions → `deploy` → Run workflow con `image_tag=<sha>`, `dataset=datasets/<snapshot>.csv.gz`, `terraform=true` y `charts` = (vacío). Crea la red, el cluster `tfvm-dev-gke` (tarda unos 10–15 min), Artifact Registry, el bucket `gs://<proyecto>-tfvm-artifacts` y las cuentas `vm-api` / `vm-trainer`.

2. **Imágenes:** re-ejecutá `ci` sobre `main` y anotá el sha publicado.

3. **Dataset al bucket:**

   ```
   gcloud auth application-default login     # solo si no lo hiciste
   .\tasks.ps1 upload-dataset -Project <proyecto>
   ```

4. **Charts y observabilidad:** `deploy` con `terraform=false`, `charts=postgres training api` y `observability=true`.

5. **Credenciales y entrenamiento en GKE.** El primer entrenamiento promueve cada versión, porque no hay vigente.

   ```
   .\tasks.ps1 gke-creds -Project <proyecto>
   .\tasks.ps1 gke-train -Model v1
   .\tasks.ps1 gke-train -Model v2
   .\tasks.ps1 gke-train -Model v2_notef
   kubectl get pods        # la API pasa a Ready cuando carga v2_notef (recarga cada 120 s)
   ```

6. **Grafana:**

   ```
   kubectl -n observability port-forward svc/kube-prom-grafana 3000:80
   $p = kubectl -n observability get secret grafana-admin -o jsonpath="{.data.admin-
   ```

```
password}"
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($p))
```

Abrí http://localhost:3000 (usuario `admin`) → dashboard **TF Vaca Muerta - API de inferencia** → Alerting.

Fase 5. Pruebas de carga (GKE)

Ver Resultados (matriz completa y cómo registrar cada corrida).

Apagar (costos)

Pausa (de un día para otro, sin perder nada)

Bajar los nodos a 0 conserva el cluster, el disco de PostgreSQL, el bucket y las imágenes. **Primero hay que desactivar el autoescalado:** si queda activo, los pods que se quedan sin nodo aparecen como pendientes y GKE vuelve a crear nodos, hasta el máximo (pasó el 11/11: tras el resize a 0 había 4 nodos corriendo).

```
gcloud container clusters update tfvm-dev-gke --zone us-central1-a --node-pool tfvm-dev-gke-pool --no-enable-autoscaling --quiet
gcloud container clusters resize tfvm-dev-gke --zone us-central1-a --node-pool tfvm-dev-gke-pool --num-nodes 0 --quiet
gcloud compute instances list --format="value(name,status)"   # no debe quedar ningún gke-tfvm-*
```

Para retomar (vuelve a los valores de Terraform: 1–4 nodos):

```
gcloud container clusters update tfvm-dev-gke --zone us-central1-a --node-pool tfvm-dev-gke-pool --enable-autoscaling --min-nodes 1 --max-nodes 4 --quiet
gcloud container clusters resize tfvm-dev-gke --zone us-central1-a --node-pool tfvm-dev-gke-pool --num-nodes 1 --quiet
```

Final (al terminar el trabajo)

```
kubectl delete pvc --all                    # discos de PostgreSQL
```

Después, destruir la infraestructura localmente:

```
cd terraform/envs/dev
terraform init -backend-config="bucket=<proyecto>-tfstate"
terraform destroy -var="project_id=<proyecto>"
```

El cluster del **laboratorio 8** (`infra-cloud-dev-gke`) es independiente: si sigue encendido también cobra. Apagalo cuando el lab esté entregado.

Problemas frecuentes

Síntoma	Causa	Solución
Pod `api` en 0/1 Ready	No hay `current.json` en el store	Entrenar (`gke-train`) y esperar la recarga, o `POST /admin/reload`
`/models` muestra `403` en `errors`	Workload Identity mal ligado	Verificar la anotación del KSA `vm-api` y el binding `workloadIdentityUser` (Terraform)
`ImagePullBackOff`	El tag no está en Artifact Registry	Usar el sha que publicó `ci`
`ErrImageNeverPull` en k3d	Falta importar la imagen	`.\tasks.ps1 k3d-images`

Síntoma	Causa	Solución
Job de training en `Error` con “sha256 no coincide”	Snapshot modificado	Regenerar con `data-build` y volver a subirlo
HPA `<unknown>`	metrics-server todavía sin datos	Esperar 1–2 min
`deploy` → `Permission denied` en bucket o IAM	Faltan roles de `ci-deployer`	Bloque 4 de `bootstrap-gcp.ps1`

Especificación de la API

Base: `http://<host>:8081`. Documentación interactiva (OpenAPI) en `/docs`. Los ejemplos se muestran con `Invoke-RestMethod` (PowerShell) y con `curl`.

> Los ejemplos 1, `/wells/{idpozo}` y `/models` son **respuestas reales** de la API desplegada en GKE el 12/11/2026 (capturas 5.5 y 2.5b; ver Resultados §3.0.1). Los ejemplos 2 a 4 muestran el formato del pedido y del error.

Resumen

Método	Ruta	Uso
POST	`/predict`	Estimación de producción mensual (m^3)
GET	`/healthz`	Readiness: 200 si el modelo por defecto está cargado, 503 si no
GET	`/livez`	Liveness: el proceso responde
GET	`/metrics`	Métricas Prometheus
GET	`/models`	Versión vigente de cada modelo, métricas y errores de carga
GET	`/models/{model}/versions`	Historial de versiones y decisiones de promoción
GET	`/wells/{idpozo}`	Datos del pozo, producción real y estimación de cada modelo mes a mes
GET	`/wells?q=texto`	Búsqueda de pozos por id, operadora o área
GET	`/wells/sample?n=200`	Ids de pozos de la población (pruebas de carga)
GET	`/stats?minutes=60`	Resumen desde PostgreSQL: predicciones (latencias, errores) y entrenamientos
POST	`/admin/reload`	Fuerza la recarga de `current.json` en la réplica (`X-Admin-Token` si `ADMIN_TOKEN` está definido)

POST /predict

Entrada (JSON)

Campo	Tipo	Obligatorio	Descripción
`model`	string	no	`v1`, `v2` o `v2_notef` (por defecto `v2_notef`)
`idpozo`	int	no*	Si se envía, la API completa edad, operadora, área, profundidad y sistema de extracción desde el dataset
`anio`, `mes`	int	no*	Mes a estimar (van juntos). Por defecto, el mes siguiente al último dato del pozo
`tef`	float 0–31	no	Días efectivos de producción

Campo	Tipo	Obligatorio	Descripción
			(v1 y v2). Si falta, se supone el mes completo
`profundidad`	float	no	m
`edad_meses`	int	no	Meses desde el inicio del pozo
`tipoextraccion`, `empresa`, `areayacimiento`	string	no	Categorías tal como figuran en los datos de la Secretaría

* Se necesita `idpozo`, o bien `anio`/`mes` junto con todas las variables del modelo elegido. Lo que se envía explícitamente **sobrescribe** lo que sale del pozo.

Salida (200)

Campo	Descripción
`model`, `version`, `use`	Modelo y versión que respondieron, y su uso (descriptivo o pronóstico)
`idpozo`, `periodo`	Pozo y mes estimado (`AAAA-MM`)
`prediccion_m3_mes`	Estimación (m^3/mes), recortada a ≥ 0
`fuera_de_rango`	`true` si el valor crudo es < 0 o > 1,5 × el máximo de entrenamiento
`categorias_no_vistas`	Variables categóricas con valores que el modelo no vio al entrenar
`entrada`	Variables efectivamente usadas
`fuentes`	De dónde salió cada dato (pozo, valores enviados, supuestos)
`referencia_mae_holdout`	MAE del modelo en el holdout fijo, como orden de magnitud del error

Errores

Código	Caso
404	`idpozo` no está en el dataset
422	Validación: faltan variables, mes fuera de rango, `anio` sin `mes`, modelo desconocido
503	Modelo no cargado (arranque o falla de carga)
500	Error inesperado (queda en los logs y en `predict_requests_total{status="500"}`)

Ejemplos

1. Solo el id del pozo (pronóstico del mes siguiente con `v2_notef`):

```
Invoke-RestMethod -Method Post -Uri http://localhost:8081/predict -ContentType
"application/json" `
  -Body '{"idpozo": 166692}'
```

Respuesta real (GKE, 12/11/2026). El pozo 166692 es del holdout: el modelo no lo vio al entrenar.


```
{
  "model": "v2_notef",
  "version": "v2_notef-20261112T001844",
```

```
  "use": "pronóstico: no usa información posterior al inicio del mes",
  "idpozo": 166692,
  "periodo": "2026-07",
  "prediccion_m3_mes": 2247.9,
  "fuera_de_rango": false,
  "categorias_no_vistas": [],
  "entrada": {"profundidad": 6641.0, "edad_meses": 21, "tipoextraccion": "Surgencia
Natural",
              "empresa": "YPF S.A.", "areayacimiento": "LA ANGOSTURA SUR I"},
  "fuentes": ["pozo 166692 (dataset)"],
  "referencia_mae_holdout": 560.8
}
```


`fuentes` dice de dónde salió cada variable. Con `v1` o `v2` y sin `tef`, agrega `"tef supuesto = días del mes"`.

2. Modelo descriptivo con TEF real del mes:

```
curl -s -X POST http://localhost:8081/predict -H "Content-Type: application/json" \
  -d '{"idpozo": 166692, "model": "v2", "anio": 2026, "mes": 6, "tef": 28}'
```

3. Pozo hipotético (todas las variables explícitas), como en la Figura 1 de la devolución:


```
{"model": "v2_notef", "anio": 2026, "mes": 3, "profundidad": 3000, "edad_meses": 6,
 "tipoextraccion": "Surgencia Natural", "empresa": "YPF S.A.", "areayacimiento": "LOMA
CAMPANA-LLL"}
```


4. Error 422 por falta de variables:


```
{"detail": "faltan variables para v2_notef: ['edad_meses', 'tipoextraccion', 'empresa',
'areayacimiento'] (envíelas o indique un idpozo)"}
```


GET /wells/{idpozo}

Reproduce el “explorador de pozos” de la devolución: producción real mes a mes y la estimación de cada modelo vigente, para analizar la declinación. Respuesta real (k3d, 11/11/2026; resumida a 2 de los 6 meses):


```
{
  "pozo": {"idpozo": 166692, "empresa": "YPF S.A.", "areayacimiento": "LA ANGOSTURA SUR I",
           "profundidad": 6641.0, "tipoextraccion": "Surgencia Natural",
           "primera_produccion": "2024-10", "ultimo_mes": "2026-06",
           "edad_censurada": false, "meses_con_datos": 6},
  "meses": [
    {"periodo": "2026-01", "prod_pet_real": 5664.0, "tef": 31.0, "tipoestado": "Extracción
Efectiva",
     "estimado_v1": 3476.3, "estimado_v2": 4151.0, "estimado_v2_notef": 3386.6},
    {"periodo": "2026-06", "prod_pet_real": 4258.0, "tef": 30.0, "tipoestado": "Extracción
Efectiva",
     "estimado_v1": 3476.3, "estimado_v2": 2967.8, "estimado_v2_notef": 2247.9}
  ],
  "versiones": {"v1": "v1-20261111T212728", "v2": "v2-20261111T212736",
                "v2_notef": "v2_notef-20261111T212741"}
}
```


La tabla completa de los 6 meses y su análisis están en Resultados §3.0.1.

GET /models

Respuesta real (GKE, 12/11/2026; se muestra un modelo de los tres):


```
{
  "default_model": "v2_notef",
  "store": "gs://unlu-ia-boyer-lab-511417-tfvm-artifacts/vm",
```

```
  "models": {
    "v2_notef": {
      "model": "v2_notef",
      "version": "v2_notef-20261112T001844",
      "use": "pronóstico: no usa información posterior al inicio del mes",
      "dataset": "vm_shale_202601_202606",
      "features": ["profundidad", "edad_meses", "tipoextraccion", "empresa",
"areayacimiento"],
      "params": {"max_depth": 8, "min_samples_leaf": 5},
      "holdout": {"mae": 560.81, "rmse": 1065.45, "r2": 0.6346},
      "cv_mae": {"mean": 572.56, "std": 28.99},
      "loaded_utc": "2026-11-12T00:21:36+00:00"
    }
  },
  "errors": {},
  "wells_dataset": "vm_shale_202601_202606",
  "wells_count": 3278
}
```


Métricas (`/metrics`)

Métrica	Tipo	Etiquetas
`predict_requests_total`	counter	`model`, `status`
`predict_latency_seconds`	histogram	`model`
`predict_value_m3`	histogram	`model`
`predict_out_of_range_total`	counter	`model`
`model_info`	gauge (1)	`model`, `version`
`model_holdout_mae_m3`	gauge	`model`
`model_reload_total`	counter	`model`, `result`
`db_write_errors_total`	counter	—

Resultados de las pruebas y análisis

> **Estado:** completo con corridas reales del 11 y 12/11/2026 (k6 `RESULT`, eventos de Kubernetes y capturas de Grafana; respaldo en `backup/gke-20261112-2021`, fuera del repo). No se escriben números que no hayan salido de una corrida.

1. Arquitectura efectivamente usada

Elemento	Valor
Proyecto / región / zona	`unlu-ia-boyer-lab-511417` / us-central1 / us-central1-a
Cluster	`tfvm-dev-gke`, zonal, canal REGULAR, versión `v1.36.4-gke.2377000` (creado por Terraform desde GitHub Actions en 11 min)
Node pool	e2-standard-2 (2 vCPU, 8 GB), disco 40 GB pd-standard, autoescalado 1–4 (nodos al inicio: 1; API, PostgreSQL y toda la observabilidad entraron en ese nodo)
Servicios desplegados	api (Deployment + HPA), postgres (StatefulSet + PVC 2 Gi), training (3 CronJobs), kube-prometheus-stack, Loki, Alloy
Recursos por pod de la API	requests 250m → **700m** (ver §3.2.3) / 384Mi, limits 1 CPU / 768Mi, 1 proceso uvicorn
HPA	2–10 réplicas, CPU 60 % de la request, estabilización 0 s al subir y 120 s al bajar
Modelo	`DecisionTreeRegressor` (scikit-learn 1.5.2) en un pipeline con one-hot; artefactos joblib de 5 a 18 KB
Dónde se entrenó	Job en GKE (CronJob `training-<modelo>` con `tune: true`); las mismas métricas se obtuvieron en la PC y en k3d
Dataset	`vm_shale_202601_202606`, sha256 `04a1ebb3…b3b2d5`, 12.321 filas / 2.212 pozos, meses 2026-01–2026-06

2. Ciclo de entrenamiento

Corridas del 11/11/2026 sobre el snapshot `vm_shale_202601_202606` (sha256 `04a1ebb3…b3b2d5`): tabla 9 del ArcGIS, más el padrón (capa 0) y las trayectorias (capa 10) descargadas ese día. scikit-learn 1.5.2, Python 3.12. Se reproducen con `.\tasks.ps1 train-all` y con el notebook.

2.1 Datos

Etapa	Registros	Pozos
Vaca Muerta · no convencional · shale	18.857	3.278
Petrolíferos	13.295	—
Petrolíferos en extracción efectiva (población)	**12.321**	**2.212**

- Meses: 2026-01 a 2026-06. `prod_pet`: mediana 698 m³/mes, media 1.285, P99 8.038 y máximo 14.579; 1,6 % de registros en cero.
- **Edad del pozo:** el padrón (`FECHA_INIC`) da fecha para el 100 % de la población.

  - En los 201 pozos que empiezan a producir dentro del semestre, la primera producción llega una mediana de **7 meses** después de `FECHA_INIC` (P10 = 5). La fecha es, por lo tanto, el **inicio de la perforación**, y la variable se interpreta como "meses desde el inicio de la perforación".
  - El P90 de 117 meses corresponde a pozos viejos que no produjeron en enero y volvieron a producir después.
- **Largo horizontal** (capa 10): cubre el 52,3 % de la población.
- Concentración de la población: YPF tiene el 66,3 % de los registros, Vista el 11,2 % y Shell el 7,0 %. Por área: Loma Campana 27,1 %, La Amarga Chica 14,8 % y Bandurria Sur 9,5 %.

2.2 Reproducción del R^2 original (punto 1 de la devolución)

Mismos 12.321 registros del notebook de Colab, `DecisionTreeRegressor(max_depth=6, min_samples_leaf=20)`.

Separación	MAE	RMSE	R^2
Por pozo, `GroupShuffleSplit(test_size=0.2, random_state=42)` (original)	703,1	1.222,1	**0,476**
Por pozo, semilla 42, árbol **sin** límites	891,9	1.681,9	0,008
Aleatoria por fila (mezcla meses de un pozo)	639,7	1.128,3	0,511

- El 0,48 se reproduce cuando se usan `min_samples_leaf=20` y `max_depth=6`.
- Un árbol sin límites sobreajusta ($R^2 \approx 0$), lo que probablemente explica la diferencia con la corrida del docente.
- Separar por fila infla el R^2 en 0,035 porque el modelo ve otros meses del mismo pozo.

2.3 Validación repetida (10 separaciones por pozo, 75/25, conjunto de desarrollo)

Versión	Modelo	MAE (m^3/mes)	RMSE	R^2
—	Predecir el promedio	1.170 ± 29	1.686 ± 72	0,00
v1	Regresión lineal	884 ± 19	1.393 ± 55	0,316 ± 0,021
v1	Árbol	706 ± 35	1.233 ± 65	0,464 ± 0,040
v2	Regresión lineal	799 ± 27	1.287 ± 70	0,415 ± 0,050
v2	Árbol	**513 ± 27**	923 ± 68	**0,699 ± 0,034**
v2_notef	Regresión lineal	820 ± 28	1.326 ± 67	0,379 ± 0,049
v2_notef	Árbol	585 ± 34	1.059 ± 76	0,604 ± 0,039

- La variabilidad entre semillas es de unos ±0,04 de R^2, así que diferencias menores entre modelos no son concluyentes con una sola separación.
- **De v1 a v2:** el MAE baja un 27 % (de 706 a 513) y la diferencia es unas 7 veces el desvío entre semillas. La edad del pozo concentra el 71 % de la importancia del árbol v2; después vienen TEF (11 %) y profundidad (8 %).
- **Costo de no usar el TEF:** quitarlo (v2 → v2_notef) sube el MAE de 513 a 585 (+14 %). Aun así, v2_notef supera claramente a v1, que sí usa el TEF.

2.4 Holdout fijo (432 pozos, 20 %) y validación temporal

Versión	MAE train	R^2 train	MAE holdout	R^2 holdout	Temporal, mismos pozos (MAE / R^2)	Temporal, pozos nuevos (MAE / R^2)
v1	654	0,526	696	0,465	652 / 0,483	751 / 0,449
v2	450	0,767	493	0,721	519 / 0,635	535 / 0,708
v2_notef	488	0,708	588	0,605	591 / 0,528	644 / 0,592

- **Validación temporal:** se entrena con enero–abril (pozos de desarrollo) y se prueba con mayo–junio. Los resultados son del mismo orden que la validación por pozo, así que el modelo no depende de ver el mes a estimar.
- **Sobreajuste:** la brecha entre train y holdout es moderada (0,05 a 0,10 de R^2).

2.5 Largo horizontal (capa 10)

Versión	MAE	R^2
v2	513 ± 27	0,699 ± 0,034
v2 + largo horizontal	515 ± 27	0,698 ± 0,033
v2_notef	585 ± 34	0,604 ± 0,039
v2_notef + largo horizontal	591 ± 37	0,601 ± 0,040

- La correlación de Spearman entre largo horizontal y producción es 0,38, pero **no mejora el modelo**.
- Motivos: solo cubre el 52 % de los pozos, y está correlacionado con la profundidad informada (ρ = 0,65; la profundidad parece incluir la rama horizontal) y con la edad (ρ = −0,41: los pozos nuevos son más largos).
- **No se incorpora.**

2.6 Versiones, tiempos y promoción

Hiperparámetros con `--tune`: grilla `max_depth` ∈ {4, 6, 8, 10, 12} × `min_samples_leaf` ∈ {5, 10, 20, 50}, con GroupKFold(5) sobre el conjunto de desarrollo.

Versión	Parámetros	MAE holdout	CV MAE	Fit (s)	Total (s)	¿Promovida?	Motivo (registrado en metadata.json)
v1 base	depth 6, leaf 20	696,5	706 ± 35	0,02	1,2	Sí	No había versión vigente
v1 `--tune`	depth 6, leaf 5	698,9	709 ± 36	0,02	4,2	No	Mejora −0,35 % < 1 %
v2 base	depth 8, leaf 20	492,7	513 ± 27	0,04	1,7	Sí	No había versión vigente
v2 `--tune`	depth 10, leaf 5	490,2	512 ± 26	0,05	7,0	No	Mejora 0,51 % < 1 % (IC 90 % [−15,8; 21,1] m^3)
v2_notef base	depth 10, leaf 20	588,4	585 ± 34	0,05	1,7	Sí	No había versión

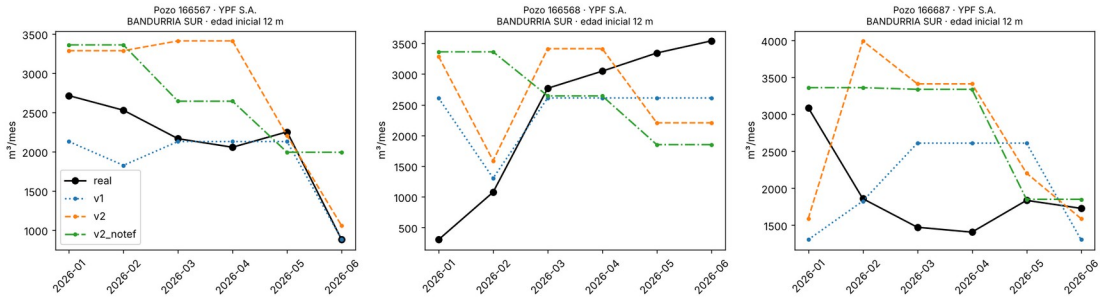
Versión	Parámetros	MAE holdout	CV MAE	Fit (s)	Total (s)	¿Promovida?	Motivo (registrado en metadata.json)
							vigente
v2_notef `--tune`	depth 8, leaf 5	560,8	573 ± 29	0,04	6,1	**Sí**	Mejora 4,69 %, IC 90 % [1,7; 54,2] m^3 > 0
v2_notef `--tune` (repetida)	depth 8, leaf 5	560,8	573 ± 29	0,04	6,1	No	Mejora 0,00 % < 1 %

- Los tiempos se midieron en el contenedor de desarrollo, no en GKE. Se completan con los Jobs de GKE (sección 1).
- Los artefactos pesan entre 5 y 18 KB.
- **Reproducibilidad entre entornos:** con el mismo snapshot (sha256 `04a1ebb3…`), `train-all -Tune` dio métricas idénticas en el contenedor de desarrollo, en la PC del autor (Windows, Python 3.12.10, ~4 s por modelo) y en un Job dentro de k3d con la imagen `training:local` (v2_notef: MAE 560,8, R^2 0,635, CV 572,6 ± 29,0; 10,7 s en total incluyendo la conexión a PostgreSQL). La corrida en k3d no se promovió (mejora 0,00 % < 1 %) y quedó registrada en la tabla `training_runs`.
- **Punto 4 de la devolución:** reentrenar exactamente la misma configuración **no** promueve una versión nueva. La evaluación es sobre el mismo holdout fijo y la versión vigente se re-evalúa con el dataset actual. Una mejora pequeña (v2 con `--tune`, 0,5 %) tampoco se promueve, porque su intervalo bootstrap incluye el cero.

2.7 Declinación (punto 3)

Pozos del holdout con los 6 meses en la población (341 pozos), cambio entre el primer y el último mes:

Modelo	Grupo	Pozos	Cambio real (mediana)	Cambio estimado (mediana)	Pozos con estimación constante
v1	Todos	341	−12,5 %	0,0 %	55,7 %
v1	Jóvenes (≤ 24 meses al inicio)	82	−30,1 %	0,0 %	35,4 %
v2	Todos	341	−12,5 %	0,0 %	49,3 %
v2	Jóvenes	82	−30,1 %	**−27,4 %**	17,1 %
v2_notef	Jóvenes	82	−30,1 %	−20,1 %	30,5 %



- Con la edad del pozo, v2 **sigue la declinación de los pozos jóvenes**: estima −27 % frente al −30 % real, cuando v1 quedaba plano en todos.
- En los pozos maduros, que declinan más lento, el árbol sigue asignando los 6 meses a la misma hoja y la estimación queda constante. Es una limitación estructural de un árbol con hojas de ≥ 5 a 20 registros.
- La figura también muestra **subidas** al inicio de la producción (pozo 166568): el pozo empieza a producir unos 6 meses después de la perforación y tarda en estabilizarse. Ningún modelo capta ese tramo.

3. Servicio de inferencia

3.0 Validación local en k3d (11/11/2026)

Antes de GKE se validó el circuito completo en la PC del autor (Windows 11, 8 GB de RAM, Docker Desktop, k3d v5.9.0 con k3s v1.35.5, 1 server + 1 agent). La API corrió con requests 100m/256Mi y HPA 1–4 (`values-k3s.yaml`), y PostgreSQL con un PVC `local-path` de 2 Gi.

Prueba	Resultado
`/healthz`	`ok` tras cargar las 3 versiones vigentes desde `/artifacts` (hostPath)
`/models`	v1 `v1-20261111T212728`, v2 `v2-20261111T212736`, v2_notef `v2_notef-20261111T212741`; sin errores de carga; 3.278 pozos indexados
Entrenamiento en el cluster	Job desde el CronJob `training-v2-notef`: mismas métricas, no promovido, run registrado en PostgreSQL
Carga (k6 en el cluster, 10 VUs, 1 min)	7.567 pedidos, **100,7 req/s**, p50 76 ms, p95 **168 ms**, p99 231 ms, máx. 3.319 ms, **0 % errores**
HPA	Escaló de 1 a 4 réplicas (el máximo) durante la prueba; al terminar la CPU bajó al 3 %
Registro de predicciones	`/stats` informó 7.568 predicciones de `v2_notef-20261111T212741` en PostgreSQL (las de k6 más una manual)

- El máximo de 3,3 s coincide con el escalado: los pods nuevos reciben tráfico mientras el nodo está saturado (k3d corre en la misma PC que k6 y Docker Desktop). En GKE se separa ese efecto midiendo con réplicas fijas.
- Durante la validación aparecieron y se corrigieron dos detalles de presentación: el JSON no declaraba `charset=utf-8` (PowerShell 5.1 mostraba mal los acentos), y la nota “tef supuesto” aparecía también en v2_notef, que no usa TEF.

3.0.1 Caso de inferencia comentado: pozo 166692 (holdout)

El pozo 166692 (YPF, La Angostura Sur I, surgencia natural, profundidad 6.641 m, primera producción 2024-10) pertenece al holdout (hash 10 < 20): ningún modelo lo vio al entrenar. `GET /wells/166692`:

Mes	Real (m^3)	TEF	v1	v2	v2_notef
2026-01	5.664	31	3.476,3	4.151,0	3.386,6
2026-02	4.678	28	3.476,3	3.319,7	3.386,6
2026-03	4.960	31	3.476,3	3.319,7	3.386,6
2026-04	4.141	29	3.476,3	3.319,7	3.386,6
2026-05	4.502	31	3.476,3	2.967,8	2.247,9
2026-06	4.258	30	3.476,3	2.967,8	2.247,9
MAE del pozo			**≈ 1.225**	**≈ 1.360**	**≈ 1.693**

`POST /predict {"idpozo": 166692}` (v2_notef, julio 2026) devolvió **2.247,9 m^3**, sin categorías no vistas ni valores fuera de rango.

- Es un pozo de alta producción (≈ 4.700 m^3/mes contra una mediana de 698) y los tres modelos lo **subestiman**: el árbol lo ubica en hojas con pozos parecidos pero menos productivos.
- **Salto escalonado:** al pasar de 18 a 19 meses de edad (mayo), v2_notef cambia de hoja y la estimación cae un 34 %, mientras la producción real baja de forma gradual. Es la limitación estructural del árbol descripta en §2.7.
- v1 queda mejor en este pozo, aunque en el holdout completo es claramente peor (MAE 696 contra 561). Un caso aislado ilustra el comportamiento pero no reemplaza la métrica global.

3.1 Metodología

- k6 dentro del cluster (`loadtest/k6-job.yaml`): la carga llega al Service y se reparte entre las réplicas.
- Carga **cerrada**: N usuarios virtuales sin pausa, es decir, N peticiones concurrentes. Subida de 15 s y meseta de 2 min.
- Cada petición es `POST /predict {"idpozo": <al azar entre 500 pozos>}` con el modelo por defecto (`v2_notef`).
- Réplicas fijas: `.\tasks.ps1 replicas -Replicas N` (quita el HPA). Autoescalado: reinstalar el chart (HPA 2–10).
- Por corrida se guarda `results\k6-<contexto>-<label>.log` y una captura del dashboard.
- k6 reserva 500m de CPU y tiene límite de 1 núcleo; por defecto reutiliza conexiones (keep-alive). Con `-NoReuse` abre una conexión por pedido (§3.3.1).
- k6 marca la corrida como fallida si se cruzan sus umbrales (p95 > 500 ms o errores > 1 %); el resultado se guarda igual.

Comandos efectivamente usados:

```
# Réplicas fijas (1 réplica el 11/11; 3 y 5 réplicas el 12/11 con requests de 700m)
foreach ($r in 1,3,5) {
  .\tasks.ps1 replicas -Replicas $r
  foreach ($v in 1,10,50,100) { .\tasks.ps1 loadtest -Vus $v -Duration 2m -Label "r$r-
```

```
vus$v" }
}
# HPA 2-10: reinstalar el chart api (deploy.yml con charts=api), esperar 2 réplicas y luego
.\tasks.ps1 loadtest -Vus 150 -Duration 4m -Label hpa-vus150              # keep-alive
.\tasks.ps1 loadtest -Vus 150 -Duration 4m -Label hpa-noreuse-b -NoReuse  # una conexión
por pedido
```

3.2 Réplicas fijas

Corridas del 11/11/2026 en GKE (1 nodo e2-standard-2 al inicio; k6 2.3.0 como Job en el mismo cluster; 2 min de meseta por corrida; `POST /predict` con el modelo por defecto). Valores de las líneas `RESULT` (`results/k6-gke_…-<label>.log`).

3.2.1 Hallazgo: sondas de salud que tumban la réplica saturada (imagen `14d68a6`)

Corrida	VUs	RPS	p50	p95	p99	Máx.	Errores
r1-vus1	1	62,2	13,5 ms	≈ 28 ms	40,7 ms	205 ms	0 %
r1-vus10	10	58,5	144 ms	297 ms	492 ms	1.937 ms	0 %
r1-vus50	50	94,7*	—	1.073 ms	1.512 ms	3.072 ms	**52,7 %**

* Incluye respuestas fallidas inmediatas (conexión rechazada), por eso el RPS “sube”.

Secuencia (eventos de Kubernetes): con 50 VUs sobre una réplica saturada, `/livez` superó el timeout de 1 s de la sonda (*context deadline exceeded*), Kubernetes **reinició el pod** (*failed liveness probe, will be restarted*) y durante el reinicio no hubo ninguna réplica disponible (*connection refused*, *i/o timeout*). El panel “Réplicas de la API” muestra el bache a 0 disponibles.

Causa: `/livez` y `/healthz` eran funciones síncronas y hacían cola en el mismo pool de threads que `/predict`; con la CPU saturada, la sonda esperaba detrás de las predicciones. Corrección (commit `2565ebb`): sondas y `/metrics` como `async def` (se atienden en el event loop), readiness con timeout de 3 s y liveness tolerante (timeout 5 s × 6 intentos ≈ 1 min). **Lección:** una sonda de liveness agresiva convierte una saturación (latencia alta) en una caída (errores), justo cuando más se necesita la réplica.

3.2.2 1 réplica, sondas corregidas (imagen `2565ebb`)

Réplicas	VUs	RPS	p50 (ms)	p95 (ms)	p99 (ms)	Máx. (ms)	Errores	CPU del pod (Grafana)
1	1	60,5	14,0	29,1	42,0	161	0 %	≈ 0,9 núcleos
1	10	62,1	142,9	262,5	374,6	642	0 %	≈ 0,9
1	50	58,0	796,4	1.200,2	1.783,4	2.338	0 %	≈ 0,9
1	100	54,1	1.704,2	2.544,2	3.218,9	3.937	0 %	≈ 0,9

- **Techo de una réplica ≈ 60 req/s.** Con un solo VU (peticiones en serie) ya se alcanza: cada predicción cuesta ≈ 15 ms de CPU, dominados por armar el DataFrame de pandas y el preprocesamiento, no por el árbol.
- Por encima del techo, más VUs solo agregan **espera en cola**: la latencia crece en proporción (ley de Little: 100 VUs ÷ 54 req/s ≈ 1,85 s, contra un p50 medido de 1,70 s).
- Con las sondas corregidas, **0 % de errores aun con 100 VUs**: la réplica atiende lento pero no se cae.

3.2.3 3 réplicas en un solo nodo (requests de 250m)

Réplicas	VUs	RPS	p50 (ms)	p95 (ms)	p99 (ms)	Máx. (ms)	Errores	CPU por pod (Grafana)
3	10	48,5	156,6	469,3	752,3	1.223	0 %	≈ 0,43
3	50	45,1	933,9	2.083,6	2.803,8	3.803	0 %	≈ 0,43
3	100	53,9	1.720,2	2.770,1	3.576,7	4.331	0 %	≈ 0,43

- **Tres réplicas rindieron menos que una.** La API pedía 250m de CPU pero usa ≈ 0,9 núcleos bajo carga: el scheduler ubicó las 3 réplicas en el único nodo (2 vCPU, compartido con la observabilidad y con k6) y, como "entraban", el autoescalador de nodos no agregó ninguno. Cada pod quedó con ≈ 0,43 núcleos (≈ 1,3 en total).
- **Corrección** (commit `4ea251c`): request de CPU de la API = **700m**, cercano al consumo real (entran 2 réplicas por nodo e2-standard-2 y los pods que no entran obligan a GKE a sumar nodos); k6 reserva 500m.
- Pendiente: repetir 3 y 5 réplicas con 700m (§3.2.4) y el HPA (§3.3).

3.2.4 3 y 5 réplicas con requests de 700m (12/11/2026, imagen `2565ebb`, chart de `f047c59`)

Réplicas	VUs	RPS	p50 (ms)	p95 (ms)	p99 (ms)	Máx. (ms)	Errores	Nodos
3	1	78,6	11,5	17,4	25,7	141	0 %	2 (2 + 1 réplicas)
3	10	**179,6**	42,3	**105,9**	156,5	622	0 %	2
3	50	165,9	278,1	457,9	572,6	943	0 %	2
3	100	157,1	592,7	921,3	1.119,0	1.637	0 %	2
5	1	57,8	14,0	32,6	50,3	341	0 %	3 (2 + 2 + 1)
5	10	231,3	17,7	135,1	209,4	790	0 %	3
5	50	**248,5**	179,2	**330,9**	472,2	1.103	0 %	3
5	100	226,4	355,3	875,4	1.177,9	2.008	0 %	3

Comparación con 1 réplica (§3.2.2), mismo número de usuarios:

VUs	1 réplica	3 réplicas (700m)	5 réplicas (700m)
10	62,1 req/s · p95 263 ms	179,6 · 106 ms (×2,9)	231,3 · 135 ms (×3,7)
50	58,0 · 1.200 ms	165,9 · 458 ms (×2,9)	248,5 · 331 ms (×4,3)
100	54,1 · 2.544 ms	157,1 · 921 ms (×2,9)	226,4 · 875 ms (×4,2)

- Con CPU real disponible, **3 réplicas rinden ≈ 3 veces** lo que una (cada pod ≈ 0,86–0,98 núcleos, captura 7.5b). Frente a §3.2.3 (las mismas 3 réplicas en un solo nodo) el throughput pasó de ≈ 50 a ≈ 180 req/s y el p95 con 100 VUs de 2,8 s a 0,9 s.
- **5 réplicas escalan, pero menos que proporcional** (×4,1–4,3 en lugar de ×5): dos nodos alojaban 2 réplicas cada uno y k6 compartía nodo con la API. Cada pod igual usaba ≈ 0,85–0,93 núcleos; el límite probable es la CPU de los nodos compartidos o el propio k6 (limitado a 1 núcleo). Queda como hipótesis: no se midió por separado.
- **Con 1 VU más réplicas no ayudan:** las peticiones son en serie y van a una réplica por vez. El 78,6 req/s con 3 réplicas frente a 60 req/s en la corrida de 1 réplica refleja que k6 ya no competía por CPU con la API (reserva de 500m).

- Con 100 VUs el p95 deja de mejorar entre 3 y 5 réplicas (921 → 875 ms): por encima de ≈ 250 req/s el sistema ya está en su techo con esta cantidad de nodos.
- `k6-r3b-vus100` y `k6-r5-vus100` terminaron como `Failed`: k6 sale con código de error cuando se cruzan los umbrales del script (p95 > 500 ms), no por errores HTTP (0 %).

3.3 Autoescalado (HPA 2–10, objetivo 60 % de CPU, requests de 700m)

Dos corridas con la misma carga (150 VUs, 4 min) partiendo de 2 réplicas. La diferencia es cómo k6 reutiliza las conexiones TCP.

	`hpa-vus150`	`hpa-noreuse-b`
Conexiones de k6	keep-alive (por defecto)	una por pedido (`-NoReuse`)
Pedidos	33.542	63.370
RPS medio (k6)	131,3	**247,7**
Pico (Grafana)	≈ 133 req/s	**295 req/s**
p50 / p95 (k6)	1.093 / 1.520 ms	**255** / 1.962 ms
Errores	0 %	0 %
Réplicas pedidas por el HPA	5	**10** (máximo)
Réplicas disponibles	5	7 (3–4 en `Pending`)
Nodos	2 → 4	4 (máximo)

Secuencia de la segunda corrida (eventos `SuccessfulRescale` de `kubectl describe hpa api`, captura 7.6b, horas reconstruidas desde su antigüedad, ±10 s):

Hora	Réplicas	Motivo
20:02:00	3 → 5	CPU sobre el objetivo
20:02:15	5 → 6	
20:02:29	6 → 8	
20:02:43	8 → **10**	máximo, ≈ 45 s después del primer escalado
≈ 20:06		termina la carga
20:07:39	10 → 4	CPU bajo el objetivo, tras la ventana de 120 s
20:07:54	4 → **2**	mínimo

3.3.1 Hallazgo: con keep-alive, las réplicas nuevas no reciben tráfico

En `hpa-vus150` el HPA subió a 5 réplicas y GKE agregó 2 nodos (captura 7.6a), pero el throughput quedó plano en ≈ 130 req/s: las 2 réplicas originales trabajaron a ≈ 0,95 núcleos y las 3 nuevas casi a 0 (captura 7.7a). k6 abre una conexión TCP por usuario virtual y la reutiliza; un Service de Kubernetes (kube-proxy) reparte **conexiones**, no pedidos, así que las 150 conexiones quedaron fijadas a los 2 pods que existían al empezar. El promedio de CPU que ve el HPA ((2 × 135 % + 3 × ≈ 0 %) / 5 ≈ 54 %) quedó bajo el 60 % y el HPA se detuvo en 5.

Con una conexión por pedido (`-NoReuse`), cada pedido se reparte entre todas las réplicas: el throughput se duplicó, el p50 bajó a la cuarta parte y el HPA fue hasta 10. El p95 más alto (1.962 ms) refleja el primer tramo con 2 réplicas y el costo de abrir una conexión por pedido.

Conclusión: el HPA solo ayuda si el balanceo reparte **pedidos**. Con clientes que mantienen conexiones largas hace falta un balanceador de capa 7 (Ingress HTTP, gateway o service mesh) o limitar la vida de las conexiones del lado del servidor.

3.3.2 Hallazgo: el tope del node pool limita al HPA (pods `Pending`)

Con el HPA en 10, 3 a 4 réplicas quedaron en `Pending` (capturas 7.9b1–3 y 7.8). Los eventos de cada pod son los mismos:

- `FailedScheduling`: *0/4 nodes are available: 4 Insufficient cpu.*
- `NotTriggerScaleUp` (cluster-autoscaler): *Pod didn't trigger scale-up: 1 max node group size reached*.

La causa es **falta de CPU reservable en los nodos** con el autoescalador de nodos ya en su máximo (4 nodos e2-standard-2): cada nodo tiene ≈ 1,9 núcleos asignables, de los que el sistema y la observabilidad usan una parte; entran ≈ 6–7 réplicas de 700m. Para 10 réplicas harían falta ≈ 6 nodos (o nodos más grandes). El mismo efecto dejó al pod de k6 en `Pending` 2,5 min (captura 7.8) hasta que el HPA bajó réplicas y liberó CPU.

3.3.3 Tiempos de reacción

- **HPA al subir:** de 3 a 10 réplicas en ≈ 45 s, en saltos cada ≈ 15 s (política por defecto: hasta 4 pods o 100 % cada 15 s, visible en `describe hpa`).
- **Disponibilidad real:** en `hpa-vus150` las réplicas pedidas tardaron ≈ 2,5 min en estar disponibles porque GKE tuvo que crear 2 nodos (≈ 70 s), descargar la imagen y cargar los modelos. El escalado de pods es rápido; el de nodos domina.
- **HPA al bajar:** espera la ventana de 120 s configurada y vuelve al mínimo en dos pasos (10 → 4 → 2 en 15 s).
- **Autoescalador de nodos al bajar:** retira los nodos sobrantes ≈ 10 min después; al hacerlo reubica pods y corta los `port-forward` (pasó con el túnel de Grafana).

3.4 Análisis

- **Saturación de una réplica:** ≈ 60 req/s por réplica (un proceso uvicorn, ≈ 15 ms de CPU por predicción, dominados por pandas y el preprocesamiento). A partir de ahí la latencia crece solo por espera en cola (ley de Little). El docente midió ≈ 200 rps por proceso con un modelo más liviano; aquí cada pedido arma un DataFrame y aplica el pipeline completo.
- **Escalado:** casi lineal hasta 3 réplicas (×2,9) y sublineal con 5 (×4,2), siempre que cada réplica tenga CPU propia. Con requests de 250m el scheduler apiló 3 réplicas en un nodo y rindieron menos que una (§3.2.3).
- **Justificación del HPA:** el request debe reflejar el consumo real (≈ 0,9 núcleos bajo carga → 700m) para que el scheduler reparta y el autoescalador de nodos actúe. Con 700m, el objetivo del 60 % (≈ 420m) dispara el escalado antes de que una réplica sature. El tope real lo pone el node pool (4 nodos).
- **Alerta `PredictLatenciaP95Alta`:** no se disparó en ninguna prueba. Usa la latencia medida **dentro** del servidor (p95 ≈ 120–320 ms), que no incluye la espera en cola antes de que la réplica tome el pedido; la latencia que percibe el cliente (k6) llegó a 1,5–2,5 s. Para producción la alerta debería basarse en la latencia medida en el balanceador/Ingress o en una sonda sintética.
- **Errores:** 0 % en todas las corridas con la configuración final. El único episodio con errores (52,7 %) fue la sonda de liveness reiniciando una réplica saturada (§3.2.1), corregido.

Reflexión crítica

Las cifras salen de Resultados (corridas del 11 y 12/11/2026).

1. Alcances

- Plataforma completa y reproducible: datos versionados → entrenamiento con validación y promoción controlada → API con trazabilidad de versión → despliegue por IaC y CI/CD con OIDC → métricas, logs y alertas → pruebas de carga y escalado.
- Tres modelos con usos explícitos: descriptivo (v1, v2) y de pronóstico (v2_notef).
- Cada predicción indica qué versión respondió, con qué datos de entrada y de dónde salieron.
- Reproducibilidad comprobada: el mismo snapshot dio las mismas métricas en la PC del autor, en k3d y en un Job en GKE (v2_notef: MAE 560,8 m^3, R^2 0,635).
- Servicio medido en GKE: ≈ 60 req/s por réplica, hasta ≈ 250–295 req/s con 5–7 réplicas, 0 % de errores con la configuración final.

2. Limitaciones del modelo

- **Ventana corta de datos:** la tabla pública cubre un semestre de 2026. Las métricas tienen varianza alta entre separaciones (por eso se informa promedio ± desvío) y no prueban que el modelo funcione en otros años o contextos de precio y actividad.
- **Declinación:** con la edad del pozo, v2 sigue la caída de los pozos jóvenes (−27 % estimado contra −30 % real), pero en los pozos maduros el árbol asigna los 6 meses a la misma hoja y la estimación queda plana (ver Resultados §2.7). Es una limitación estructural de un árbol sobre variables casi constantes por pozo. Alternativas a probar: la producción del mes anterior como variable (requiere la serie histórica) o curvas de Arps por pozo.
- **Edad del pozo:** sale de `FECHA_INIC` del padrón, que resultó ser el inicio de la **perforación**: la producción empieza unos 7 meses después (mediana). El desfase varía entre pozos (P10 = 5 meses) y agrega ruido. Además, ningún modelo capta la subida de los primeros meses de producción.
- **Variables faltantes:** etapas de fractura, arena y agua bombeadas (Adjunto IV, otro dataset), presiones, choke. El largo horizontal (capa 10) se evaluó y no mejora el modelo: cubre el 52 % de los pozos y su información ya está en la profundidad y la edad.
- **TEF:** v1 y v2 lo necesitan y solo se conoce al cerrar el mes. Si no se informa, la API supone el mes completo, lo que sobrestima en meses con paradas.
- **Categorías nuevas:** una operadora o área que no estaba en el entrenamiento se codifica como “ninguna categoría” (la respuesta lo marca en `categorias_no_vistas`).

2b. Limitaciones de la plataforma (halladas en las pruebas)

- **Una réplica es un solo proceso** y cada predicción cuesta ≈ 15 ms de CPU, sobre todo en armar el DataFrame de pandas y en el preprocesamiento. El techo por réplica es ≈ 60 req/s; por encima, solo crece la espera en cola.
- **Las sondas de salud** síncronas y con timeout de 1 s reiniciaron una réplica saturada y la saturación se volvió caída (52,7 % de errores). Se corrigió (sondas `async`, timeouts tolerantes), pero muestra lo frágil que es la configuración por defecto bajo carga.
- **El request de CPU** tiene que reflejar el consumo real. Con 250m (contra ≈ 900m reales) el scheduler apiló 3 réplicas en un nodo y rindieron menos que una. Con 700m escalan.
- **El balanceo de un Service reparte conexiones, no pedidos.** Con clientes que mantienen conexiones abiertas (keep-alive), las réplicas que agrega el HPA casi no reciben tráfico y el HPA se frena.

- **El tope del node pool** (4 nodos e2-standard-2) limita al HPA: con 10 réplicas pedidas, 3–4 quedaron `Pending` por falta de CPU. Los nodos nuevos tardan ≈ 70 s más la descarga de la imagen y la carga de los modelos: el escalado de pods es rápido, el de nodos domina.
- **La alerta de latencia** mide dentro del servidor (p95 ≤ 320 ms) y no ve la espera en cola que percibe el cliente (p95 de hasta 2,5 s): no se disparó en ninguna prueba.
- **Prometheus no tiene almacenamiento persistente:** al bajar los nodos se perdieron las métricas del primer día (quedaron capturas, los resultados de k6 y un respaldo exportado del segundo día).

3. Implicancias éticas

- **Datos:** públicos y agregados por pozo, sin datos personales. Igual se cita la fuente, la fecha de descarga y el filtro aplicado (manifest), y no se redistribuye el dataset completo de la Secretaría, solo el subconjunto filtrado.
- **Sesgos:** la población está dominada por pocas operadoras y áreas: YPF tiene el 66 % de los registros y Loma Campana el 27 %. El modelo aprende mejor esos casos. Incluir la operadora puede reproducir diferencias operativas o de reporte entre empresas en lugar de diferencias geológicas. Pozos parados o con producción cero se excluyen por diseño (extracción efectiva), así que el modelo no sirve para estimar paradas.
- **Uso:** es una estimación académica con error medio del orden de 560 m^3/mes (v2_notef, holdout) frente a una producción mediana de 698 m^3/mes. No debe usarse para decisiones de inversión, regalías, reservas ni seguridad sin validación de ingeniería. La API lo dice en su descripción y devuelve el MAE de referencia en cada respuesta.
- **Transparencia:** cada versión documenta datos, parámetros, métricas y por qué fue o no promovida. Las predicciones quedan registradas con versión y latencia.

4. Qué falta para producción real

Área	Falta
Datos	Ingesta automática mensual, validación de esquema y de rangos (p. ej. Great Expectations), monitoreo de *drift* de entradas y salidas
Modelo	Serie histórica multi-año, validación temporal estricta, intervalos de predicción (no solo un valor), modelos que sigan la declinación
Inferencia	Preprocesamiento sin pandas por pedido (o un formato compilado del modelo), varios workers por réplica, pedidos por lote para estimar muchos pozos a la vez
Tráfico	Ingress o gateway HTTP (capa 7) que reparta pedidos y no conexiones, con TLS; límite de vida de las conexiones; *rate limiting*
Escalado	Requests calibrados con mediciones (como los 700m), node pool más grande o aprovisionamiento automático de nodos, nodos precalentados o réplicas mínimas según la demanda esperada
Seguridad	Autenticación de la API (API key u OAuth), `ADMIN_TOKEN` desde Secret Manager, red privada para GKE, formato de modelo sin ejecución de código (ONNX o `skops`)
Observabilidad	Alertas sobre la latencia que percibe el cliente (balanceador o sonda sintética), almacenamiento persistente para Prometheus, Alertmanager con notificaciones, SLOs y *runbooks*
Operación	Cluster regional o al menos multi-zona, PostgreSQL

Área	Falta
	gestionado con backups (Cloud SQL), retención y archivado de la tabla de predicciones (380.947 filas en dos días de pruebas)
Entrega	Entornos dev/prod separados con aprobación manual, despliegues canary o blue-green de versiones del modelo, rollback automático si empeoran las métricas en línea
Costos	Apagado automático fuera de horario, nodos Spot para entrenamiento, política de limpieza del Artifact Registry (llegó a 7,95 GB por publicar en cada push), presupuestos y alertas de facturación

5. Continuidad (Curso 4, IA generativa)

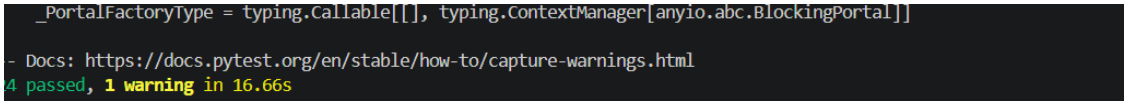
La misma plataforma (API, registro de versiones, PostgreSQL, observabilidad y despliegue) sirve de base para:

- un asistente que responda preguntas en lenguaje natural sobre los pozos (“¿cuánto produjo el pozo X?”, “¿qué estima el modelo para julio?”) llamando a `/wells` y `/predict` como herramientas (MCP);
- un modelo abierto servido en el cluster (p. ej. con Ollama), con las mismas métricas y alertas.

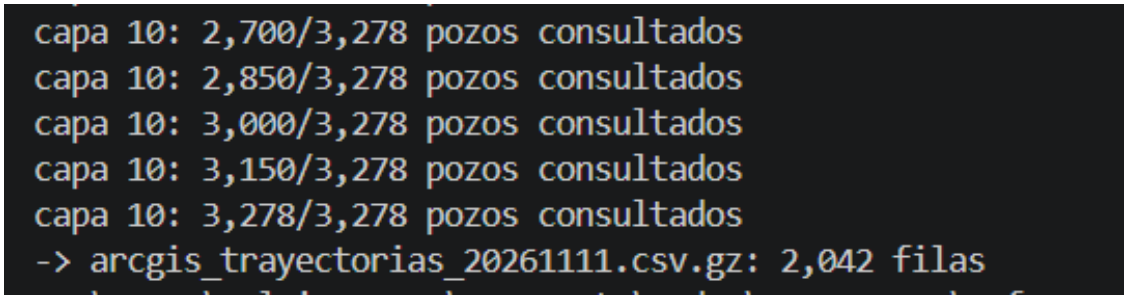
Anexo: evidencias de la ejecución

Capturas de la ejecución real, agrupadas con la misma lógica que los entregables del Laboratorio 8 (CI, Terraform y despliegue, pods y `/predict`, Grafana, carga y escalado, estado final) más las propias del trabajo (datos, entrenamiento, k3d). Son un respaldo de lo informado en los capítulos anteriores, no una sección aparte del análisis.

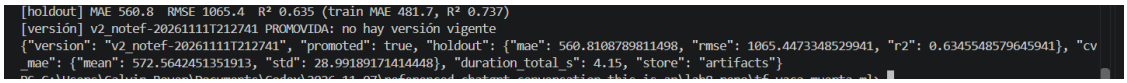
1. Datos y entrenamiento local (PC, Windows)



Evidencia 1.1. `.\tasks.ps1 test`: 24 tests pasados

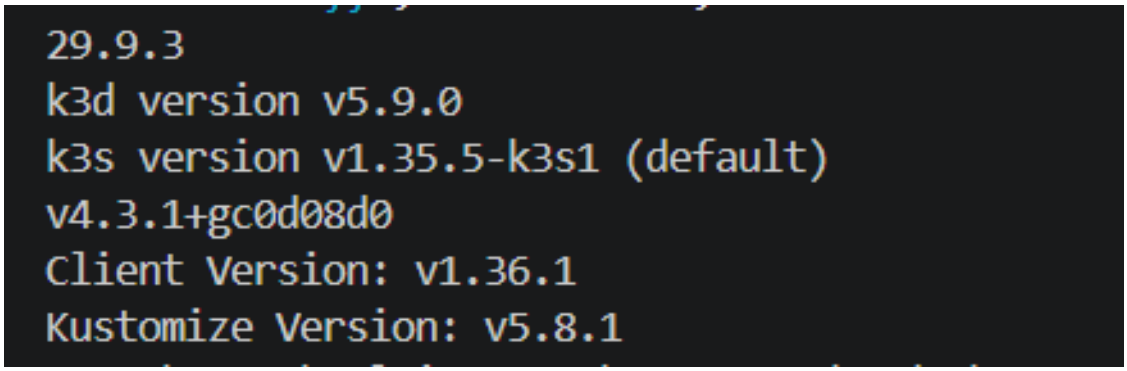


Evidencia 1.2. Descarga de la capa 10 (trayectorias): 2.042 filas para 3.278 pozos



Evidencia 1.3. `train-all -Tune`: v2_notef MAE 560,8, R^2 0,635, promovida

2. Kubernetes local (k3d)



Evidencia 2.1. Docker 29.9.3, k3d 5.9.0, Helm 4.3.1, kubectl 1.36.1

```
NAME                STATUS   ROLES           AGE   VERSION
k3d-tfvm-agent-0    Ready    <none>          10s   v1.35.5+k3s1
k3d-tfvm-server-0   Ready    control-plane   15s   v1.35.5+k3s1
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversatio
```

Evidencia 2.2. Cluster `tfvm`: server y agent `Ready`

```
NAME                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE
service/api          ClusterIP   10.43.76.235   <none>        8081/TCP   16s
service/kubernetes   ClusterIP   10.43.0.1      <none>        443/TCP    8m15s
service/postgres     ClusterIP   10.43.61.206   <none>        5432/TCP   64s

NAME                                   REFERENCE         TARGETS              MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/api   Deployment/api   cpu: <unknown>/60%   1         4         0          15s

NAME                           SCHEDULE    TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
cronjob.batch/training-v1      0 6 5 * *   <none>     True      0        <none>          16s
cronjob.batch/training-v2      0 6 5 * *   <none>     True      0        <none>          16s
cronjob.batch/training-v2-notef   0 6 5 * *   <none>     True      0        <none>          16s

NAME                                STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS
   AGE
persistentvolumeclaim/data-postgres-0   Bound    pvc-75ffe214-2ef0-4629-a8ce-5b83e78eef55   2Gi        RWO            local-path     <unset>
   63s
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml>
```

Evidencia 2.3a. Services, HPA, 3 CronJobs suspendidos, PVC `Bound`

```
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-cha
NAME                   READY   STATUS    RESTARTS   AGE
api-6cb6478cc9-gltwj   1/1     Running   0          77s
postgres-0             1/1     Running   0          2m5s
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-cha
```

Evidencia 2.3b. `api` y `postgres-0` en `Running`

```
[versión] v2_notef-20261111T215730 no promovida: mejora relativa 0.00% < 1% (MAE 560.8 vs vigente 560.8)
[db] run registrado en training_runs
{"version": "v2_notef-20261111T215730", "promoted": false, "holdout": {"mae": 560.8108789811498, "rmse": 1065.4473348529941, "r2": 0.6345
548579645941}, "cv_mae": {"mean": 572.5642451351913, "std": 28.99189171414448}, "duration_total_s": 10.716, "store": "/artifacts"}
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml>
```

Evidencia 2.4. Job en k3d: mismas métricas, no promovido, run en PostgreSQL

```
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml> .\tasks.ps1 l
oadtest -Vus 10 -Duration 1m
configmap/k6-script created
job.batch/k6-vus10-185811 created
Esperando el Job k6-vus10-185811 (en otra terminal: kubectl get hpa -w)...
> kubectl wait --for=condition=complete job/k6-vus10-185811 --timeout=30m
job.batch/k6-vus10-185811 condition met
RESULT {"label":"vus10-185811","vus":10,"duration":"1m","model":"default","requests":7567,"rps":100.7,"p50_ms":76.1,"p90_ms":140.9,"p95_m
s":167.7,"p99_ms":231.1,"max_ms":3319.2,"error_rate":0}
Log guardado en results\k6-k3d-tfvm-vus10-185811.log
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml> S
```

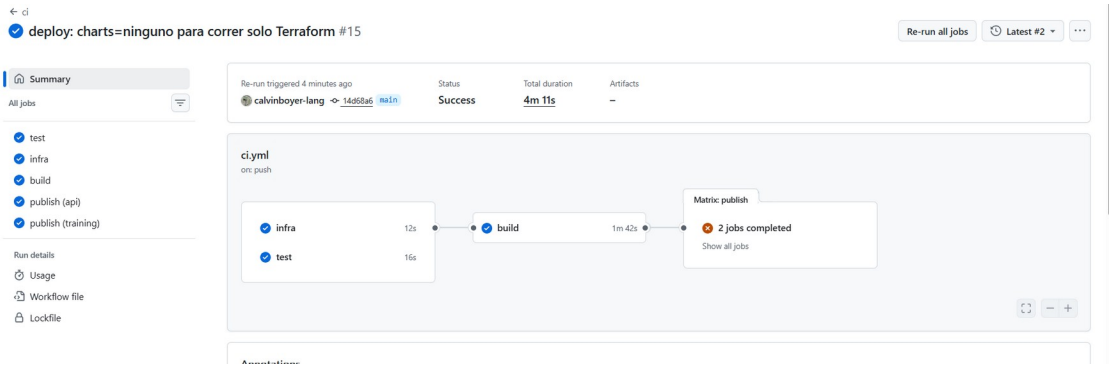
Evidencia 2.5a. k6 en el cluster: 100,7 req/s, p95 168 ms, 0 % errores

```
pa,pods; Invoke-RestMethod http://localhost:8081/stats | ConvertTo-Json -Depth 5
NAME                                    REFERENCE         TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/api   Deployment/api   cpu: 3%/60%   1         4         4          19m

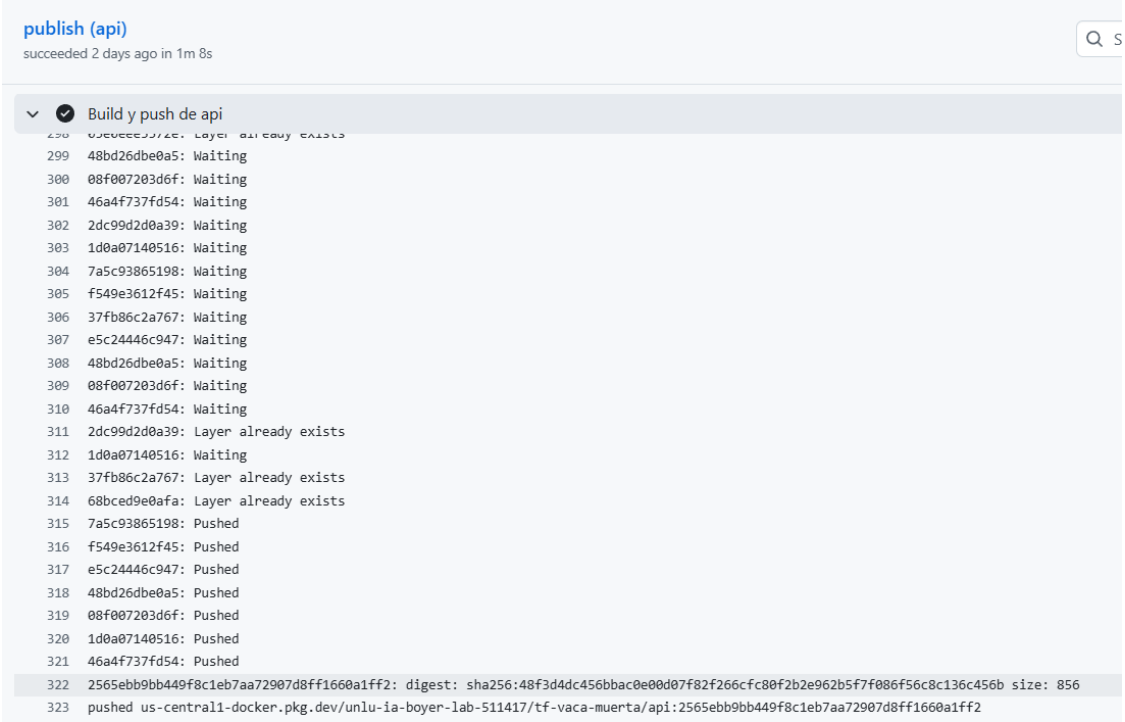
NAME                                READY   STATUS      RESTARTS   AGE
pod/api-7fc8987df8-2nxc5            1/1     Running     0          117s
pod/api-7fc8987df8-kpgbs            1/1     Running     0          117s
pod/api-7fc8987df8-qr7zl            1/1     Running     0          8m26s
pod/api-7fc8987df8-wbnm7            1/1     Running     0          117s
pod/k6-vus10-185811-9ljt4           0/1     Completed   0          2m21s
pod/postgres-0                      1/1     Running     0          20m
pod/train-v2-notef-1111185709-9qx7q   0/1     Completed   0          3m24s
{
    "ventana_min":  60,
    "predicciones":  [
                         {
                             "model":  "v2_notef",
                             "version":  "v2_notef-20261111T212741",
                             "n":  7568,
```

Evidencia 2.5b. HPA 1 → 4 réplicas y `/stats` con 7.568 predicciones en PostgreSQL

3. CI (equivalente al entregable 1.1 del lab)

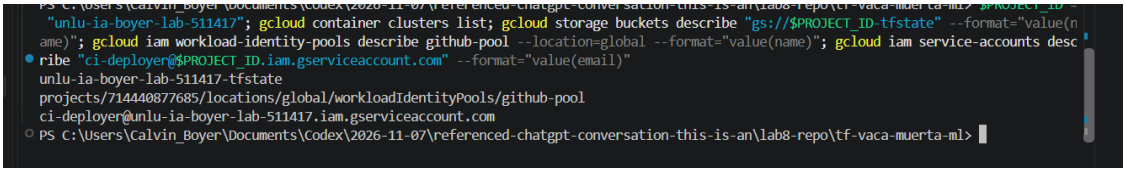


Evidencia 3.1. `ci` #15 (commit `14d68a6`) en Success: test, infra, build, publish (api) y publish (training)



Evidencia 3.2. Log de **publish (api)**: `pushed …/tf-vaca-muerta/api:2565ebb9bb449f8c1eb7aa72907d8ff1660a1ff2` con su digest sha256 (la imagen desplegada en GKE)

4. GCP y Terraform (equivalente al 2.1)



Evidencia 4.1a. Bucket de tfstate, pool `github-pool` y `ci-deployer` existentes; sin clusters GKE prendidos



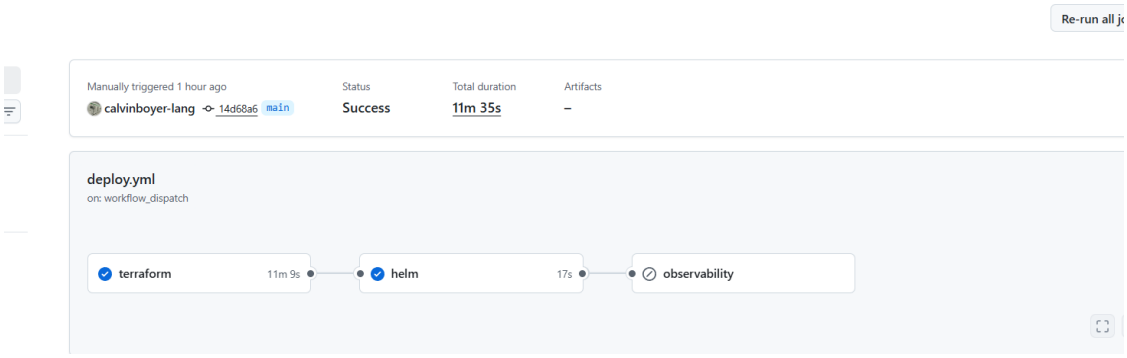
Evidencia 4.1b. Provider OIDC `github-provider-tfvm` creado

```
uerta-ml" --format="value(etag)"
Updated IAM policy for project [unlu-ia-boyer-lab-511417].
rol roles/compute.admin -> ci-deployer
Updated IAM policy for project [unlu-ia-boyer-lab-511417].
rol roles/container.admin -> ci-deployer
Updated IAM policy for project [unlu-ia-boyer-lab-511417].
rol roles/artifactregistry.admin -> ci-deployer
Updated IAM policy for project [unlu-ia-boyer-lab-511417].
rol roles/iam.serviceAccountUser -> ci-deployer
Updated IAM policy for project [unlu-ia-boyer-lab-511417].
rol roles/storage.admin -> ci-deployer
Updated IAM policy for project [unlu-ia-boyer-lab-511417].
rol roles/iam.serviceAccountAdmin -> ci-deployer
Updated IAM policy for serviceAccount [ci-deployer@unlu-ia-boyer-lab-511417.iam.gserviceaccount.com].
BwZgB2OcR2I=
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml>
```

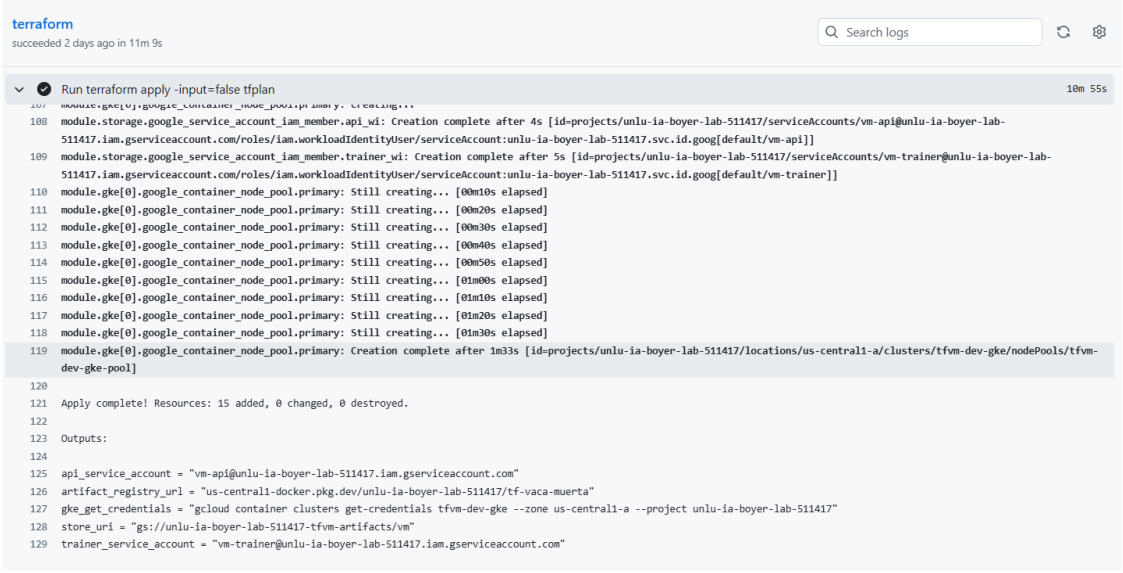
Evidencia 4.1c. Roles de `ci-deployer` y binding de Workload Identity del repo

Name	Value	Last updated
GCP_DEPLOY_SERVICE_ACCOUNT	ci-deployer@unlu-ia-boyer-lab-511417.iam.gserviceac...	now
GCP_PROJECT_ID	unlu-ia-boyer-lab-511417	2 minutes ago
GCP_REGION	us-central1	now
GCP_SERVICE_ACCOUNT	ci-deployer@unlu-ia-boyer-lab-511417.iam.gserviceac...	1 minute ago
GCP_WORKLOAD_IDENTITY_PROVIDER	projects/714440877685/locations/global/workloadIde...	1 minute ago

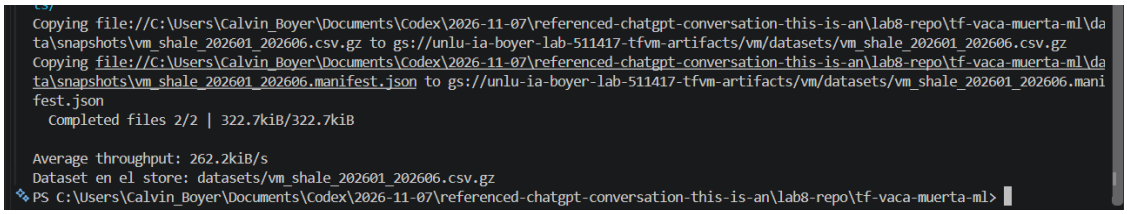
Evidencia 4.1d. Las 5 variables de GitHub Actions



Evidencia 4.2. `deploy` #1: terraform 11 min 9 s y helm en Success

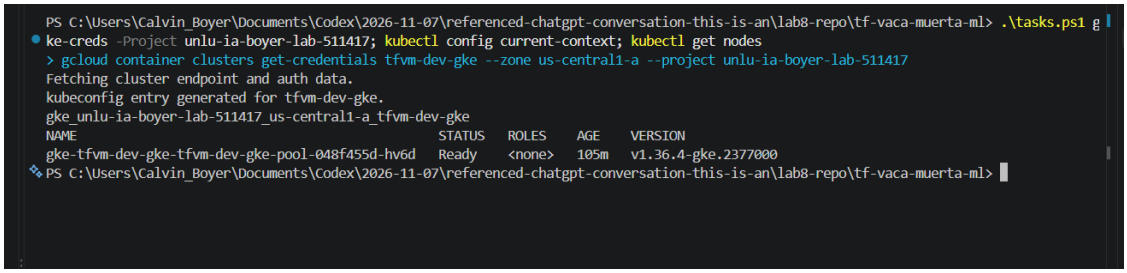


Evidencia 4.2b. `terraform apply` del deploy #1: `Apply complete! Resources: 15 added, 0 changed, 0 destroyed.`; bindings de Workload Identity creados después del cluster; node pool en 1 min 33 s; outputs (cuentas, registro, bucket)

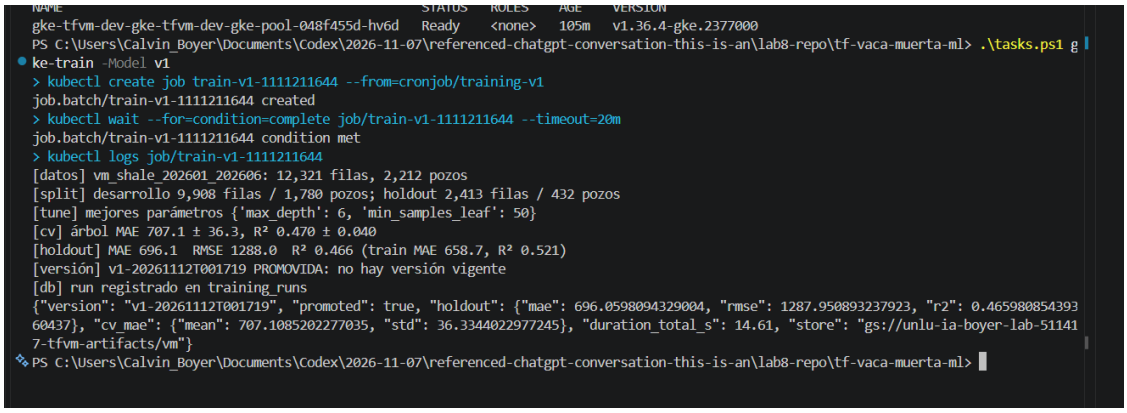


Evidencia 4.3. Snapshot y manifest subidos al bucket

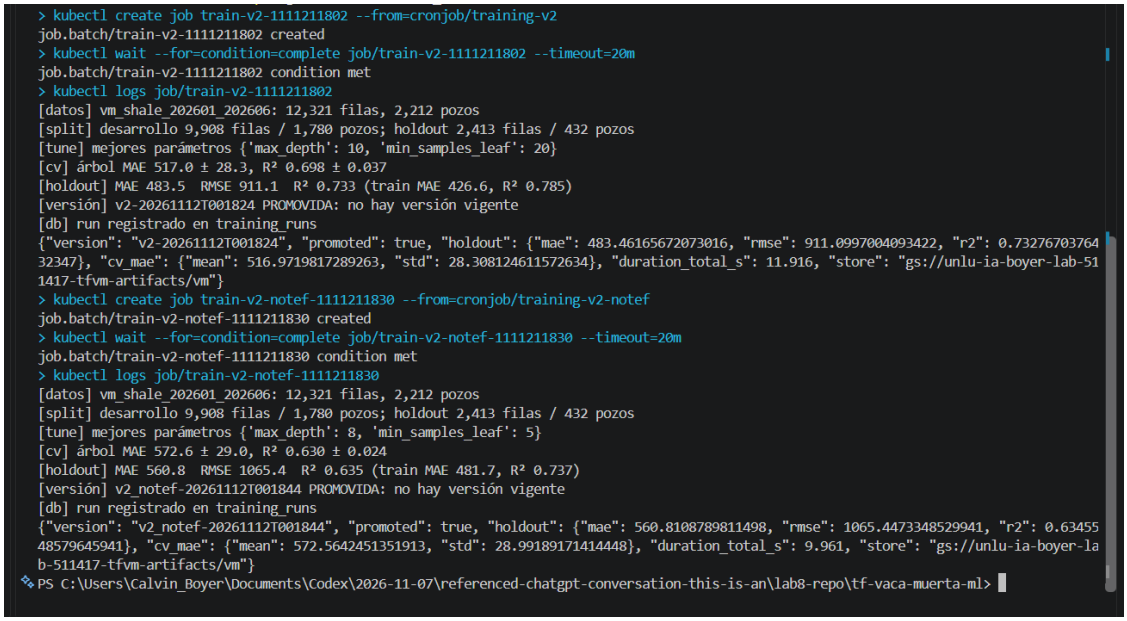
5. GKE: entrenamiento, pods y `/predict` (equivalente al 3.1)



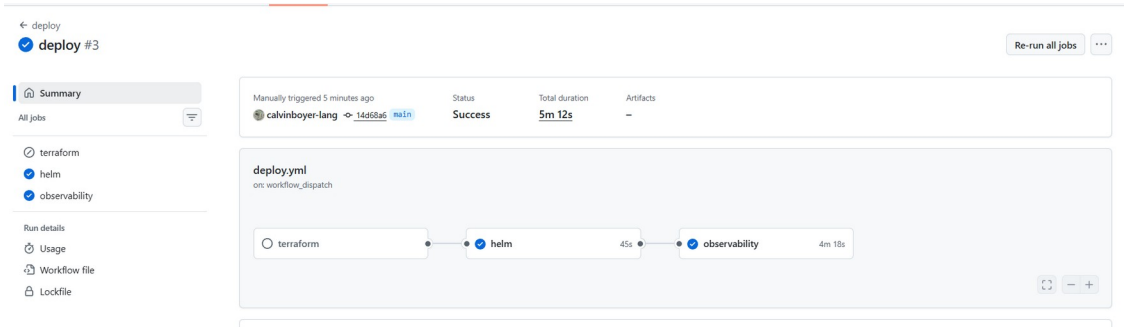
Evidencia 5.1. Contexto GKE y nodo `Ready` (v1.36.4-gke.2377000)



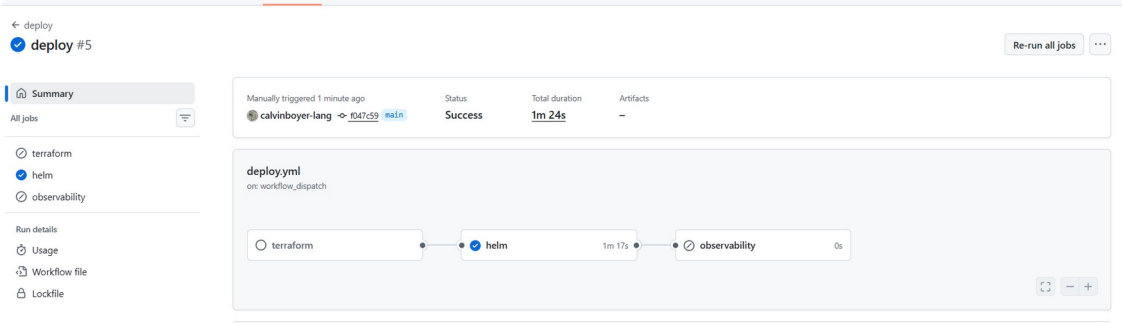
Evidencia 5.2a. Job de entrenamiento v1 en GKE: lee/escribe el bucket, promovida



Evidencia 5.2b. v2 y v2_notef entrenadas y promovidas en GKE



Evidencia 5.3. `deploy` #3: helm y observability en Success



Evidencia 5.3b. `deploy` #5: API con requests de 700m (imagen `2565ebb`)

```
NAME                                                STATUS   ROLES    AGE    VERSION
gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-hv6d    Ready    <none>   117m   v1.36.4-gke.2377000
NAME                                READY   STATUS      RESTARTS   AGE
pod/api-5b64dcfcf4-x9tsd            1/1     Running     0          5m31s
pod/api-5b64dcfcf4-zd477            1/1     Running     0          5m46s
pod/postgres-0                      1/1     Running     0          13m
pod/train-v1-1111211644-qwh9c       0/1     Completed   0          10m
pod/train-v2-1111211802-nhj2c       0/1     Completed   0          8m58s
pod/train-v2-notef-1111211830-2dv9n 0/1     Completed   0          8m29s

NAME                                       REFERENCE          TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/api    Deployment/api     cpu: 1%/60%   2         10        2          5m47s
NAME                                                   READY   STATUS    RESTARTS   AGE
alloy-d957d894c-75fq4                                  2/2     Running   0          93s
kube-prom-grafana-7cf78cd6fc-kcxt9                     3/3     Running   0          4m25s
kube-prom-kube-prometheus-operator-55d5fb7c46-pvnjt    1/1     Running   0          4m25s
kube-prom-kube-state-metrics-7d678c658-jnmh7           1/1     Running   0          4m26s
kube-prom-prometheus-node-exporter-mjfrz               1/1     Running   0          4m26s
loki-0                                                 2/2     Running   0          2m27s
loki-gateway-fd955697c-wrfds                           1/1     Running   0          2m27s
prometheus-kube-prom-kube-prometheus-prometheus-0      2/2     Running   0          4m16s
```

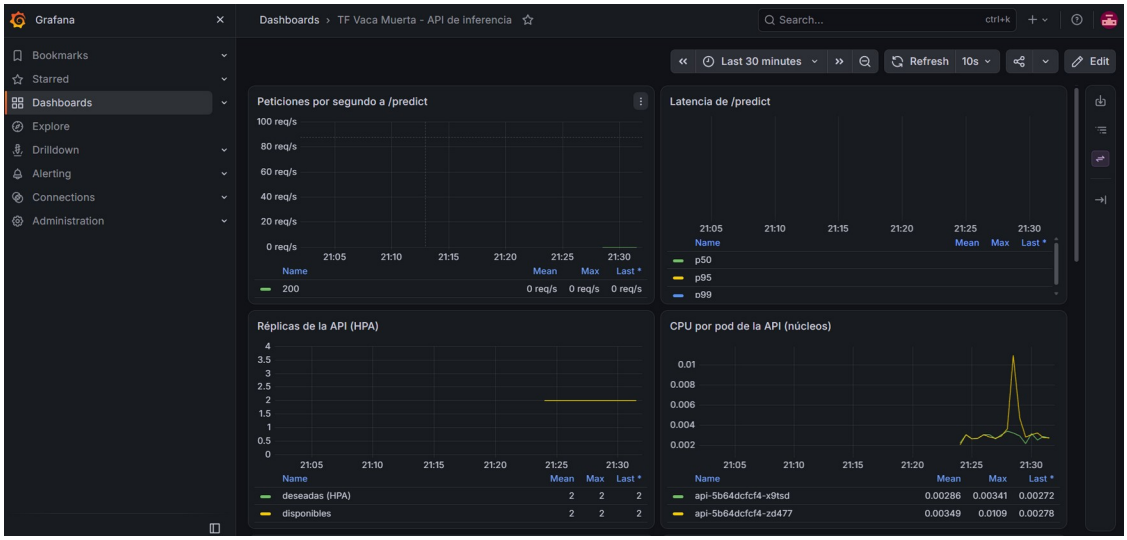
Evidencia 5.4. Pods de la API, PostgreSQL, HPA 2/10 y stack de observabilidad

```
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo> kubectl get pods -o custom-colu
mns="NAME:.metadata.name,READY:.status.containerStatuses[0].ready,STATUS:.status.phase,RESTARTS:.status.containerStatuses[0].restartCount
,NODE:.spec.nodeName"; Invoke-RestMethod -Method Post -Uri http://localhost:8081/predict -ContentType "application/json" -Body (@{idpozo=
166692} | ConvertTo-Json) | ConvertTo-Json -Depth 4
NAME                READY   STATUS    RESTARTS   NODE
api-7c7f6d7f55-4b95f   true    Running   0          gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-mzm7
api-7c7f6d7f55-px6s4   true    Running   0          gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-bvs4
postgres-0             true    Running   0          gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-mzm7
{
    "model":  "v2_notef",
    "version":  "v2_notef-20261112T001844",
    "use":  "pronóstico: no usa información posterior al inicio del mes",
    "idpozo":  166692,
    "periodo":  "2026-07",
    "prediccion_m3_mes":  2247.9,
    "fuera_de_rango":  false,
    "categorias_no_vistas":  [

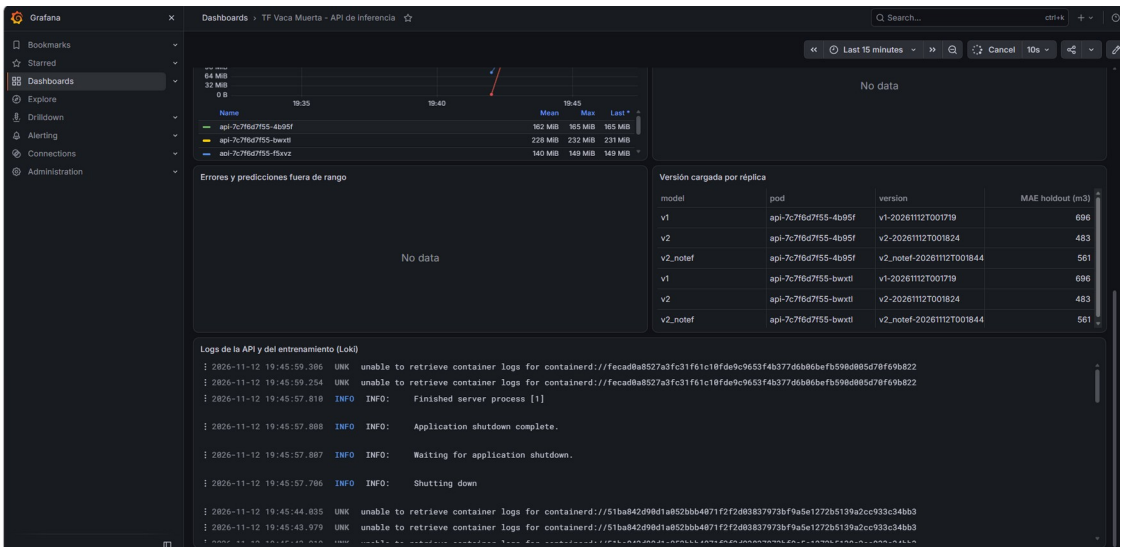
                             ],
    "entrada":  {
                    "profundidad":  6641.0,
                    "edad_meses":  21,
                    "tipoextraccion":  "Surgencia Natural",
                    "empresa":  "YPF S.A.",
                    "areayacimiento":  "LA ANGOSTURA SUR I"
                },
    "fuentes":  [
                    "pozo 166692 (dataset)"
                ],
    "referencia_mae_holdout":  560.8
}
```

Evidencia 5.5. Comando visible, encabezado de la tabla, 2 réplicas de la API y `postgres-0` en `Running` con su nodo; `/predict` responde `v2_notef-20261112T001844` (2.247,9 m^3)

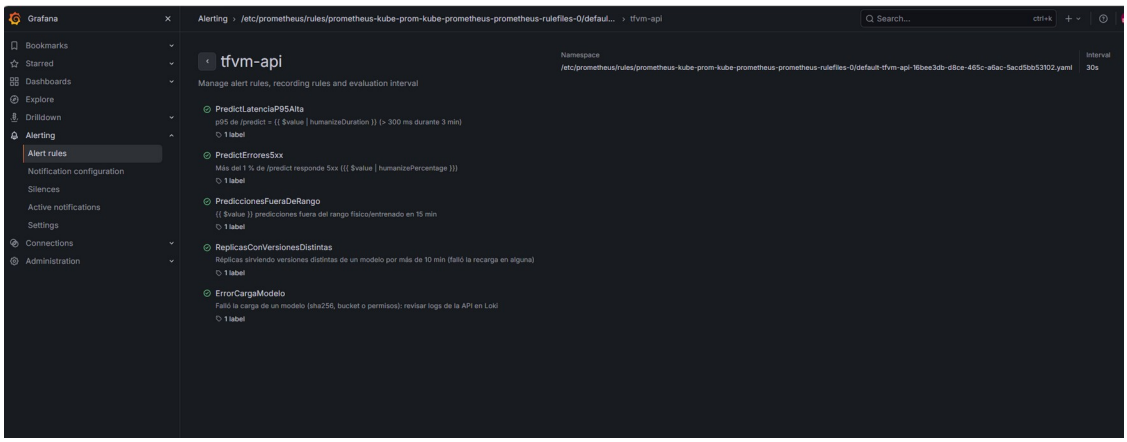
6. Grafana base, sin carga (equivalente al 4.1)



Evidencia 6.1. Dashboard “TF Vaca Muerta – API de inferencia” con 2 réplicas, CPU por pod; tráfico mínimo (2 consultas manuales, no k6)



Evidencia 6.2. Tabla “Versión cargada por réplica” (trazabilidad modelo/pod/versión/MAE) y logs de la API en Loki (apagado de réplicas al bajar el HPA)



Evidencia 6.3. Grupo `tfvm-api` con las 5 reglas cargadas en Prometheus (evaluación cada 30 s), todas en estado normal

7. Carga y escalado (equivalente al 5.1 y 5.2)



Evidencia 7.1. 1 réplica: techo ≈ 60 req/s, CPU ≈ 0,9 núcleos, bache de disponibles a 0

```
15m         Normal    Started                 pod/k6-r1-vus10-8fzrj                Container started
12m         Normal    Completed               job/k6-r1-vus10                      Job completed
12m         Normal    Scheduled               pod/k6-r1-vus50-ffjhw                Successfully assigned default/k6-r1-vus50-ffjhw
to gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-hv6d
12m         Normal    SuccessfulCreate        job/k6-r1-vus50                      Created pod: k6-r1-vus50-ffjhw
12m         Normal    Pulled                  pod/k6-r1-vus50-ffjhw                Container image "grafana/k6:2.3.0" already
present on machine and can be accessed by the pod
12m         Normal    Created                 pod/k6-r1-vus50-ffjhw                Container created
12m         Normal    Started                 pod/k6-r1-vus50-ffjhw                Container started
11m         Warning   Unhealthy               pod/api-5b64dcfcf4-x9tsd             Readiness probe failed: Get
"http://10.44.1.21:8081/healthz": read tcp 10.44.1.1:58692->10.44.1.21:8081: read: connection reset by peer
11m         Normal    Killing                 pod/api-5b64dcfcf4-x9tsd             Container api failed liveness probe, will be
restarted
11m         Normal    Created                 pod/api-5b64dcfcf4-x9tsd             Container created
11m         Normal    Pulled                  pod/api-5b64dcfcf4-x9tsd             Container image
"us-central1-docker.pkg.dev/unlu-ia-boyer-lab-511417/tf-vaca-muerta/api:14d68a6daf426c5a26a4d8eeec0981a330fc7a13" already present on
machine and can be accessed by the pod
11m         Normal    Started                 pod/api-5b64dcfcf4-x9tsd             Container started
11m         Warning   Unhealthy               pod/api-5b64dcfcf4-x9tsd             Readiness probe failed: Get
"http://10.44.1.21:8081/healthz": dial tcp 10.44.1.21:8081: connect: connection refused
10m         Warning   Unhealthy               pod/api-5b64dcfcf4-x9tsd             Liveness probe failed: Get
"http://10.44.1.21:8081/livez": context deadline exceeded (Client.Timeout exceeded while awaiting headers)
10m         Warning   BackoffLimitExceeded    job/k6-r1-vus50                      Job has reached the specified backoff limit
```

Evidencia 7.2a. Eventos: liveness con timeout → reinicio del pod; Job k6 con backoff

```
NAME                          READY   STATUS      RESTARTS      AGE
api-5b64dcfcf4-x9tsd          1/1     Running     1 (11m ago)   30m
k6-r1-vus1-ntkbz              0/1     Completed   0             17m
k6-r1-vus10-8fzrj             0/1     Completed   0             15m
k6-r1-vus50-ffjhw             0/1     Error       0             12m
postgres-0                    1/1     Running     0             38m
train-v1-1111211644-qwh9c     0/1     Completed   0             35m
train-v2-1111211802-nhj2c     0/1     Completed   0             33m
train-v2-notef-1111211830-2dv9n   0/1     Completed   0             33m
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"
time="2026-11-12T00:40:40Z" level=warning msg="Request Failed" error="Post \"http://api:8081/predict\": dial: i/o timeout"

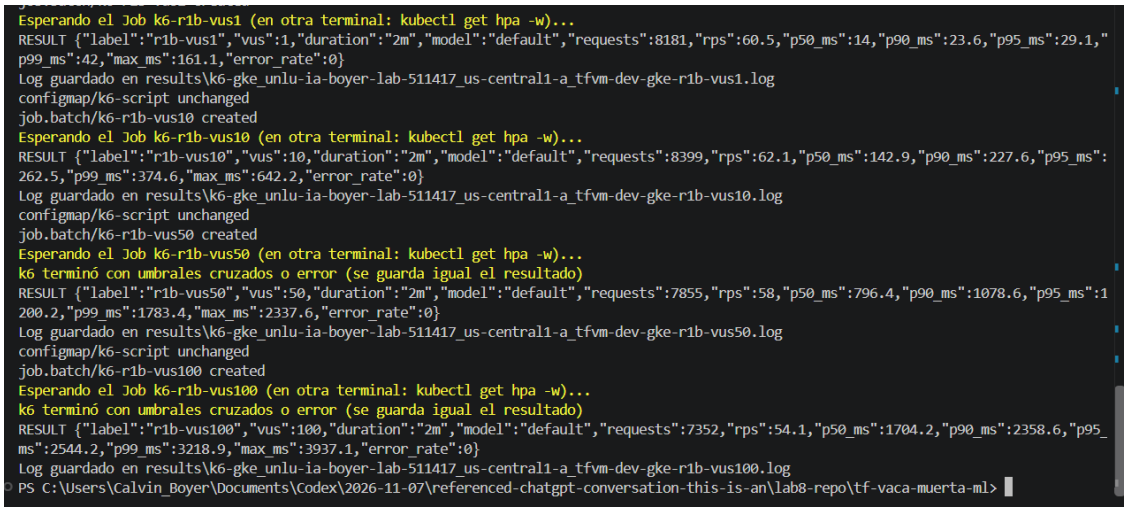
RESULT {"label":"r1-vus50","vus":50,"duration":"2m","model":"default","requests":12812,"rps":94.7,"p50_ms":0,"p90_ms":947.9,"p95_ms":1073
.2,"p99_ms":1512.2,"max_ms":3072.4,"error_rate":0.5267}
time="2026-11-12T00:41:21Z" level=error msg="thresholds on metrics 'http_req_duration{endpoint:predict}, http_req_failed' have been cross
ed"
```

Evidencia 7.2b. `RESTARTS 1` y log de k6 con `i/o timeout`: 52,7 % de errores

```
esults\k6-gke_*-r1-vus1.log | Select-String RESULT

RESULT {"label":"r1-vus1","vus":1,"duration":"2m","model":"default","requests":8398,"rps":62.2,"p50_ms":13.5,"p90_ms":23.2,"p95_ms":28.
,"p99_ms":40.7,"max_ms":204.5,"error_rate":0}
```

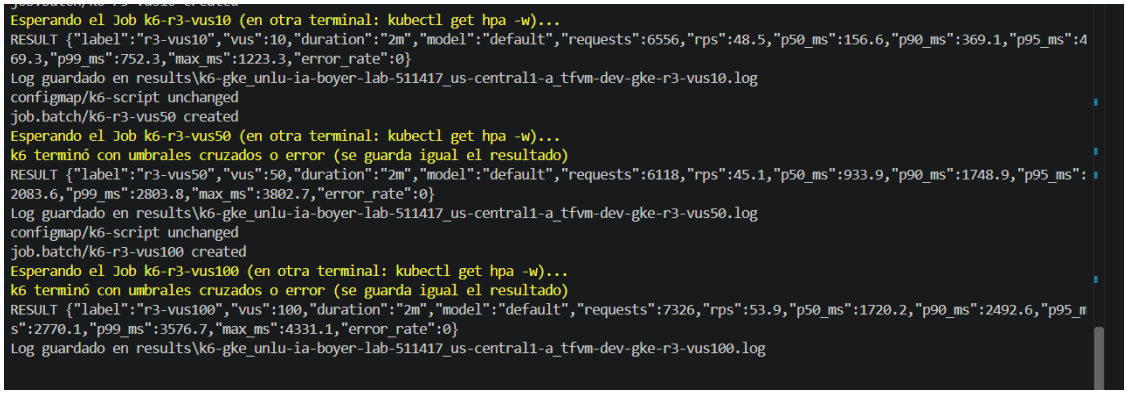
Evidencia 7.3a. 1 réplica, 1 VU: 62,2 req/s



Evidencia 7.3b. 1 réplica, 1–100 VUs con sondas corregidas: 0 % errores



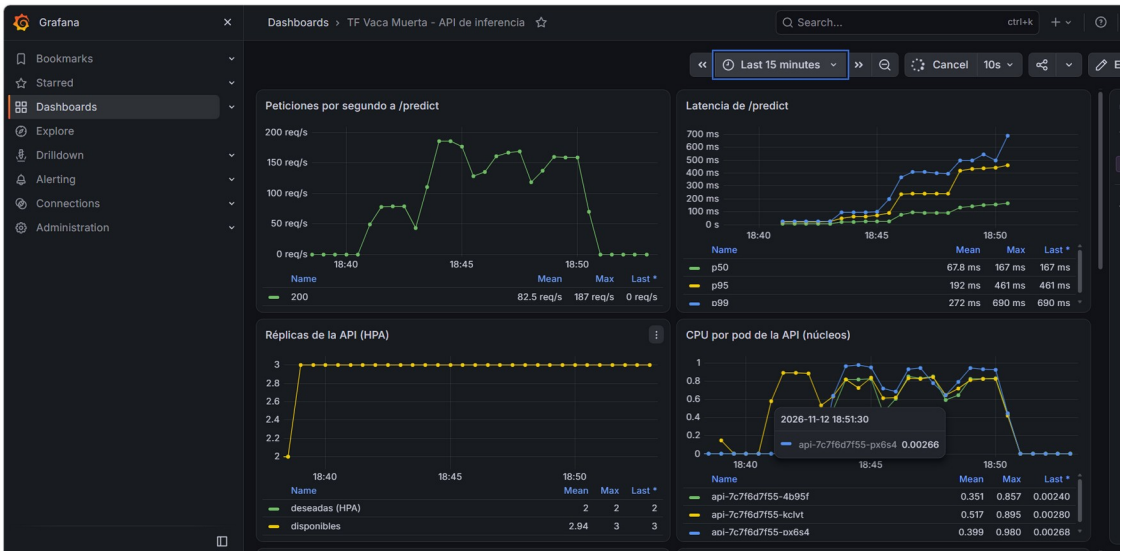
Evidencia 7.4a. 3 réplicas en un nodo: ≈ 0,43 núcleos por pod, latencia peor



Evidencia 7.4b. Resultados k6 de 3 réplicas (requests 250m)

```
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo> Set-Location C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml; .\tasks.ps1 replicas -Replicas 3; kubectl get pods -o custom-columns="NAME:.metadata.name,READY:.status.containerStatuses[0].ready,STATUS:.status.phase,NODE:.spec.nodeName"
> kubectl scale deployment/api --replicas=3
deployment.apps/api scaled
> kubectl rollout status deployment/api --timeout=5m
Waiting for deployment "api" rollout to finish: 2 of 3 updated replicas are available...
deployment "api" successfully rolled out
> kubectl get deploy/api -o wide
NAME   READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES
                                SELECTOR
api    3/3     3            3           21h   api          us-central1-docker.pkg.dev/unlu-ia-boyer-lab-511417/tf-vaca-muerta/api:2565ebb9bb449f8c1eb7aa72907d8ff1660a1ff2   app=api
NAME                   READY   STATUS    NODE
api-7c7f6d7f55-4b95f   true    Running   gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-mzm7
api-7c7f6d7f55-kclvt   true    Running   gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-mzm7
api-7c7f6d7f55-px6s4   true    Running   gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-bvs4
postgres-0             true    Running   gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-mzm7
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml>
```

Evidencia 7.5a. 3 réplicas con 700m repartidas en 2 nodos (comando visible)



Evidencia 7.5b. Dashboard con 3 réplicas: hasta 187 req/s, cada pod ≈ 0,9 núcleos

```
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo> Set-Location C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml; Get-Content results\k6-gke_*-r3b-*.log | Select-String RESULT

RESULT {"label":"r3b-vus1","vus":1,"duration":"2m","model":"default","requests":10612,"rps":78.6,"p50_ms":11.5,"p90_ms":14.8,"p95_ms":17.4,"p99_ms":25.7,"max_ms":141.2,"error_rate":0}
RESULT {"label":"r3b-vus10","vus":10,"duration":"2m","model":"default","requests":24259,"rps":179.6,"p50_ms":42.3,"p90_ms":87.3,"p95_ms":105.9,"p99_ms":156.5,"max_ms":622.1,"error_rate":0}
RESULT {"label":"r3b-vus100","vus":100,"duration":"2m","model":"default","requests":21302,"rps":157.1,"p50_ms":592.7,"p90_ms":829.8,"p95_ms":921.3,"p99_ms":1119,"max_ms":1637.4,"error_rate":0}
RESULT {"label":"r3b-vus50","vus":50,"duration":"2m","model":"default","requests":22439,"rps":165.9,"p50_ms":278.1,"p90_ms":406.7,"p95_ms":457.9,"p99_ms":572.6,"max_ms":942.5,"error_rate":0}
```

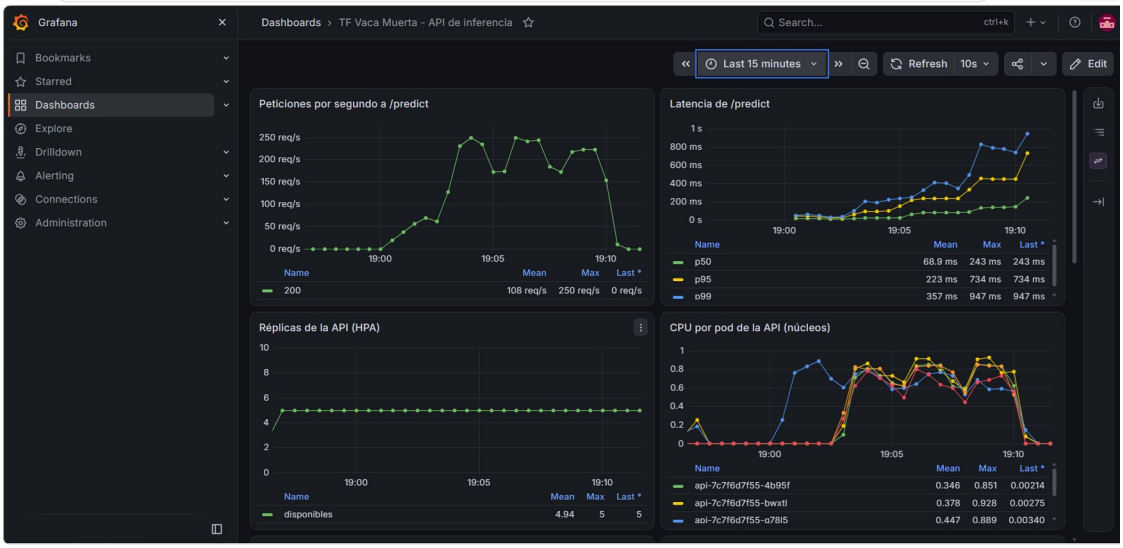
Evidencia 7.5c. k6, 3 réplicas: 78,6 / 179,6 / 165,9 / 157,1 req/s, 0 % errores

```
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml> Set-Location
C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml; kubectl scale de
ployment/api --replicas=5; Start-Sleep 5; kubectl get nodes; kubectl get pods -o custom-columns="NAME:.metadata.name,READY:.status.contai
nerStatuses[0].ready,STATUS:.status.phase,NODE:.spec.nodeName"
deployment.apps/api scaled
NAME                                                  STATUS   ROLES    AGE   VERSION
gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-bvs4      Ready    <none>   31m   v1.36.4-gke.2377000
gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-mzm7      Ready    <none>   32m   v1.36.4-gke.2377000
gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-qs5d      Ready    <none>   33m   v1.36.4-gke.2377000
NAME                READY   STATUS      NODE
api-7c7f6d7f55-4b95f   true    Running     gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-mzm7
api-7c7f6d7f55-bwxtl   false   Running     gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-qs5d
api-7c7f6d7f55-g78l5   false   Running     gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-bvs4
api-7c7f6d7f55-kclvt   true    Running     gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-mzm7
api-7c7f6d7f55-px6s4   true    Running     gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-bvs4
k6-r3b-vus1-cqcxk      false   Succeeded   gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-bvs4
k6-r3b-vus10-ndhgf     false   Succeeded   gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-qs5d
k6-r3b-vus100-b8d9m    false   Failed      gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-bvs4
k6-r3b-vus50-gdtkj     false   Succeeded   gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-bvs4
postgres-0             true    Running     gke-tfvm-dev-gke-tfvm-dev-gke-pool-048f455d-mzm7
```

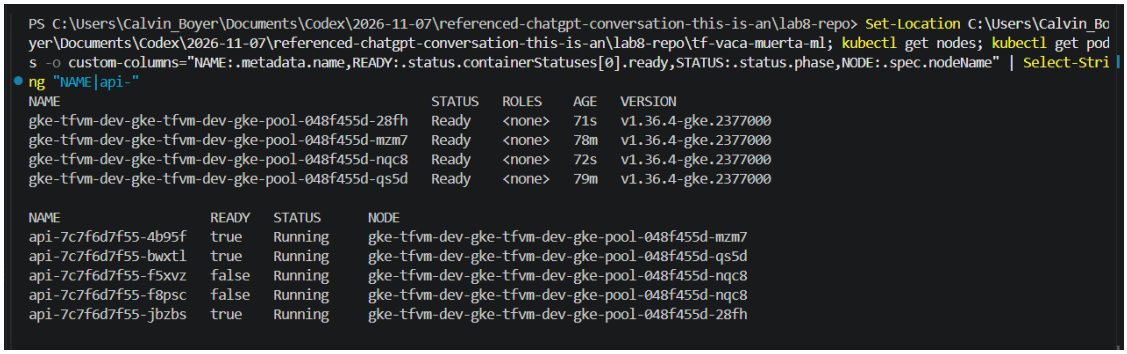
Evidencia 7.5d. 5 réplicas en 3 nodos (2 + 2 + 1), sin `Pending`

```
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml> Set-Location
C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml; kubectl rollout
status deployment/api --timeout=3m; foreach ($v in 1,10,50,100) { .\tasks.ps1 loadtest -Vus $v -Duration 2m -Label "r5-vus$v" }
deployment "api" successfully rolled out
configmap/k6-script unchanged
job.batch/k6-r5-vus1 created
Esperando el Job k6-r5-vus1 (en otra terminal: kubectl get hpa -w)...
RESULT {"label":"r5-vus1","vus":1,"duration":"2m","model":"default","requests":7814,"rps":57.8,"p50_ms":14,"p90_ms":25.9,"p95_ms":32.6,"p
99_ms":50.3,"max_ms":340.8,"error_rate":0}
Log guardado en results\k6-gke_unlu-ia-boyer-lab-511417_us-central1-a_tfvm-dev-gke-r5-vus1.log
configmap/k6-script unchanged
job.batch/k6-r5-vus10 created
Esperando el Job k6-r5-vus10 (en otra terminal: kubectl get hpa -w)...
RESULT {"label":"r5-vus10","vus":10,"duration":"2m","model":"default","requests":31238,"rps":231.3,"p50_ms":17.7,"p90_ms":104.5,"p95_ms":
135.1,"p99_ms":209.4,"max_ms":790,"error_rate":0}
Log guardado en results\k6-gke_unlu-ia-boyer-lab-511417_us-central1-a_tfvm-dev-gke-r5-vus10.log
configmap/k6-script unchanged
job.batch/k6-r5-vus50 created
Esperando el Job k6-r5-vus50 (en otra terminal: kubectl get hpa -w)...
RESULT {"label":"r5-vus50","vus":50,"duration":"2m","model":"default","requests":33581,"rps":248.5,"p50_ms":179.2,"p90_ms":280,"p95_ms":3
30.9,"p99_ms":472.2,"max_ms":1103.4,"error_rate":0}
Log guardado en results\k6-gke_unlu-ia-boyer-lab-511417_us-central1-a_tfvm-dev-gke-r5-vus50.log
configmap/k6-script unchanged
job.batch/k6-r5-vus100 created
Esperando el Job k6-r5-vus100 (en otra terminal: kubectl get hpa -w)...
k6 terminó con umbrales cruzados o error (se guarda igual el resultado)
RESULT {"label":"r5-vus100","vus":100,"duration":"2m","model":"default","requests":30720,"rps":226.4,"p50_ms":355.3,"p90_ms":720.1,"p95_m
s":875.4,"p99_ms":1177.9,"max_ms":2008.4,"error_rate":0}
Log guardado en results\k6-gke_unlu-ia-boyer-lab-511417_us-central1-a_tfvm-dev-gke-r5-vus100.log
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml>
```

Evidencia 7.5e. k6, 5 réplicas: 57,8 / 231,3 / 248,5 / 226,4 req/s, 0 % errores



Evidencia 7.5f. Dashboard con 5 réplicas: hasta 250 req/s

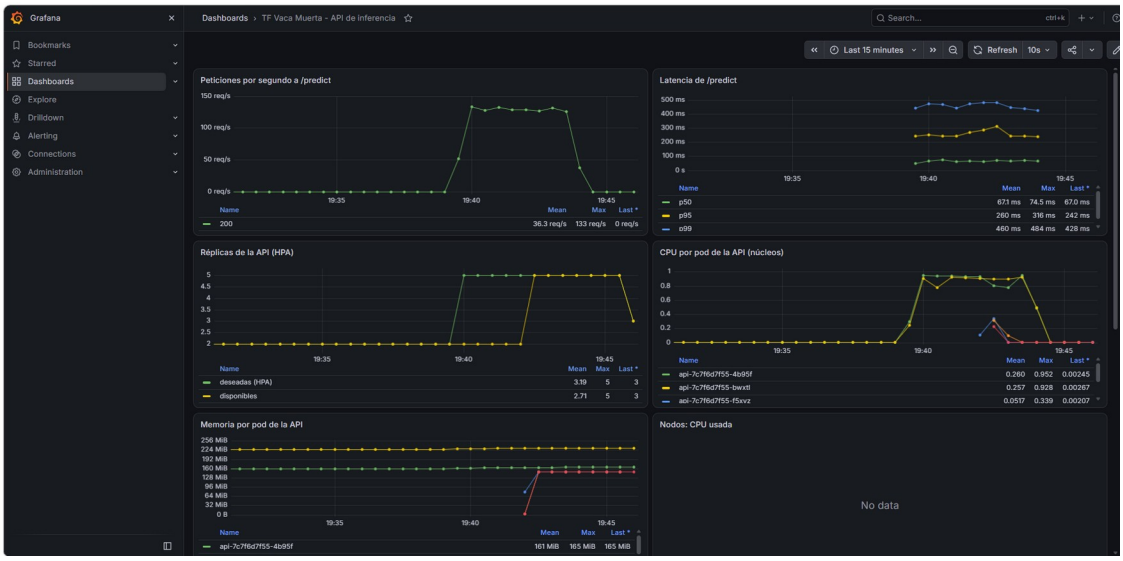


Evidencia 7.6a. Prueba con keep-alive: HPA en 5 réplicas y 2 nodos nuevos creados por GKE

```
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo> kubectl get hpa api; kubectl d
scribe hpa api | Select-String -Pattern "SuccessfulRescale|Events|Type|----"
api     Deployment/api   cpu: 0%/60%   2         10        2         57m

      - Type: Pods     Value: 4    Period: 15 seconds
      - Type: Percent  Value: 100  Period: 15 seconds
      - Type: Percent  Value: 100  Period: 15 seconds
  Type           Status  Reason           Message
  ----           ------  ------           -------
Events:
  Type     Reason                 Age                From                       Message
  ----     ------                 ----               ----                       -------
  Normal   SuccessfulRescale      55m                horizontal-pod-autoscaler  New size: 2; reason: cpu resource utilization
(percentage of request) below target
  Normal   SuccessfulRescale      32m                horizontal-pod-autoscaler  New size: 5; reason: cpu resource utilization
(percentage of request) above target
  Normal   SuccessfulRescale      26m                horizontal-pod-autoscaler  New size: 3; reason: cpu resource utilization
(percentage of request) below target
  Normal   SuccessfulRescale      22m                horizontal-pod-autoscaler  New size: 9; reason: cpu resource utilization
(percentage of request) above target
  Normal   SuccessfulRescale      10m                horizontal-pod-autoscaler  New size: 3; reason: cpu resource utilization
(percentage of request) above target
  Normal   SuccessfulRescale      10m (x2 over 23m)  horizontal-pod-autoscaler  New size: 5; reason: cpu resource utilization
(percentage of request) above target
  Normal   SuccessfulRescale      9m45s              horizontal-pod-autoscaler  New size: 6; reason: cpu resource utilization
(percentage of request) above target
  Normal   SuccessfulRescale      9m31s              horizontal-pod-autoscaler  New size: 8; reason: cpu resource utilization
(percentage of request) above target
  Normal   SuccessfulRescale      9m17s              horizontal-pod-autoscaler  New size: 10; reason: cpu resource utilization
(percentage of request) above target
  Normal   SuccessfulRescale      4m21s (x2 over 27m)  horizontal-pod-autoscaler  New size: 4; reason: cpu resource utilization
(percentage of request) below target
  Normal   SuccessfulRescale      4m6s (x3 over 26m)   horizontal-pod-autoscaler  New size: 2; reason: cpu resource utilization
(percentage of request) below target
```

Evidencia 7.6b. `kubectl describe hpa api`: eventos `SuccessfulRescale` de todas las pruebas (2 → 5 → 3 → 9 → 3 → 5 → 6 → 8 → 10 → 4 → 2) con su antigüedad y la política de subida (4 pods o 100 % cada 15 s)



Evidencia 7.7a. HPA con keep-alive: 5 réplicas pero el tráfico queda en ~130 req/s; las réplicas nuevas casi sin CPU

```
RESULT {"label":"hpa-vus150","vus":150,"duration":"4m","model":"default","requests":33542,"rps":131.3,"p50_ms":1093.2,"p90_ms":1388.4,"p
95_ms":1519.8,"p99_ms":1862.6,"max_ms":2608.9,"error_rate":0}
```

Evidencia 7.7b. RESULT: 131,3 req/s, p95 1.520 ms, 0 % errores



Evidencia 7.9a. Primer intento sin keep-alive (interrumpido al relanzar el comando): HPA pide 9, 7 disponibles

```
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml> $pend = kube
tl get pods --field-selector=status.phase=Pending -o name; if ($pend) { $pend; foreach ($p in $pend) { kubectl describe $p | Select-Obje
t -Last 15 } } else { "No hay pods en Pending" }; kubectl get hpa api
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml> $pend = kube
tl get pods --field-selector=status.phase=Pending -o name; if ($pend) { $pend; foreach ($p in $pend) { kubectl describe $p | Select-Obje
t -Last 15 } } else { "No hay pods en Pending" }; kubectl get hpa api
pod/api-7c7f6d7f55-82hms
pod/api-7c7f6d7f55-rdrfn
pod/api-7c7f6d7f55-vshlk
pod/api-7c7f6d7f55-xkdvk
  kube-api-access-hn5cm:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
  QoS Class:                 Burstable
  Node-Selectors:            <none>
  Tolerations:               node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

  Events:
    Type     Reason             Age   From               Message
    ----     ------             ----  ----               -------
    Warning  FailedScheduling   13s   default-scheduler  0/4 nodes are available: 4 Insufficient cpu. no new claims to deallocate, preemp
  ion: 0/4 nodes are available: 4 No preemption victims found for incoming pod.
    Normal   NotTriggerScaleUp  13s   cluster-autoscaler  Pod didn't trigger scale-up: 1 max node group size reached
    kube-api-access-9gjwc:
      Type:                    Projected (a volume that contains injected data from multiple sources)
      TokenExpirationSeconds:  3607
      ConfigMapName:           kube-root-ca.crt
      Optional:                false
      DownwardAPI:             true
    QoS Class:                 Burstable
    Node-Selectors:            <none>
```

Evidencia 7.9b1. 4 réplicas `Pending`: `Insufficient cpu` y `max node group size reached`; HPA en 10

```
PS C:\Users\Calvin_Boyer\Documents\Codex\2026-11-07\referenced-chatgpt-conversation-this-is-an\lab8-repo\tf-vaca-muerta-ml> $pend = kubec
tl get pods --field-selector=status.phase=Pending -o name; if ($pend) { $pend; foreach ($p in $pend) { kubectl describe $p | Select-Objec
t -Last 15 } } else { "No hay pods en Pending" }; kubectl get hpa api
  kube-api-access-9gjwc:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   Burstable
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason             Age   From               Message
  ----     ------             ----  ----               -------
  Warning  FailedScheduling   28s   default-scheduler  0/4 nodes are available: 4 Insufficient cpu. no new claims to deallocate, preempt
ion: 0/4 nodes are available: 4 No preemption victims found for incoming pod.
  Normal   NotTriggerScaleUp  28s   cluster-autoscaler  Pod didn't trigger scale-up: 1 max node group size reached
  kube-api-access-9bnm5:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   Burstable
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
```

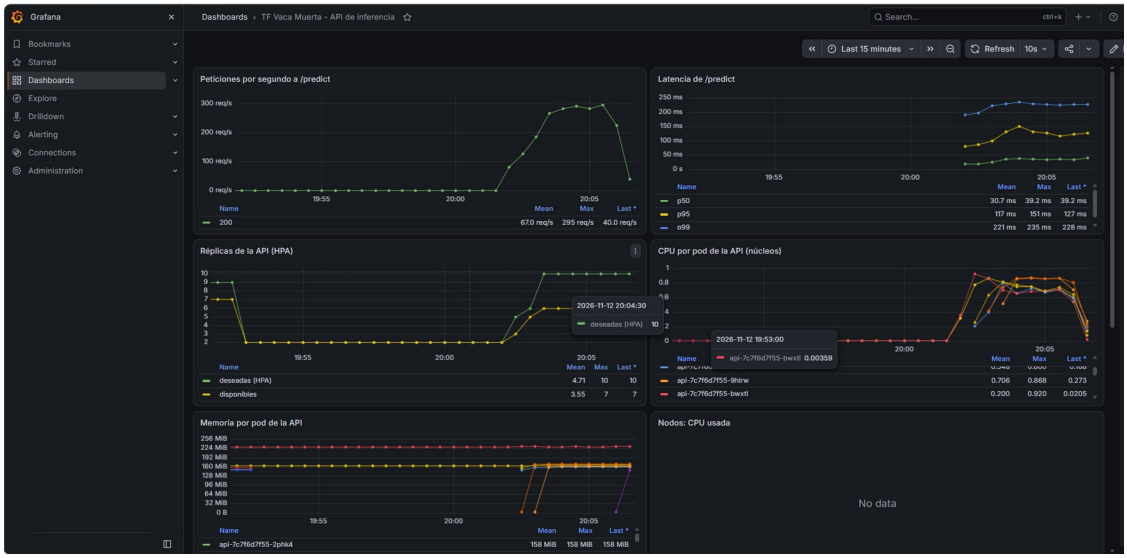
Evidencia 7.9b2. (continuación)

```
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason             Age   From               Message
  ----     ------             ----  ----               -------
  Warning  FailedScheduling   16s   default-scheduler  0/4 nodes are available: 4 Insufficient cpu. no new claims to deallocate, preempt
ion: 0/4 nodes are available: 4 No preemption victims found for incoming pod.
  Normal   NotTriggerScaleUp  15s   cluster-autoscaler  Pod didn't trigger scale-up: 1 max node group size reached
  kube-api-access-m82jg:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   Burstable
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason             Age   From               Message
  ----     ------             ----  ----               -------
  Warning  FailedScheduling   31s   default-scheduler  0/4 nodes are available: 4 Insufficient cpu. no new claims to deallocate, preempt
ion: 0/4 nodes are available: 4 No preemption victims found for incoming pod.
  Normal   NotTriggerScaleUp  31s   cluster-autoscaler  Pod didn't trigger scale-up: 1 max node group size reached
NAME   REFERENCE        TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
api    Deployment/api   cpu: 118%/60%   2         10        10         48m
```

Evidencia 7.9b3. (continuación)


```
RESULT {"label":"hpa-noreuse-b","vus":150,"duration":"4m","no_reuse":true,"model":"default","requests":63370,"rps":247.7,"p50_ms":255,"p
90_ms":1704.3,"p95_ms":1962.3,"p99_ms":2392.5,"max_ms":2941.6,"error_rate":0}
```

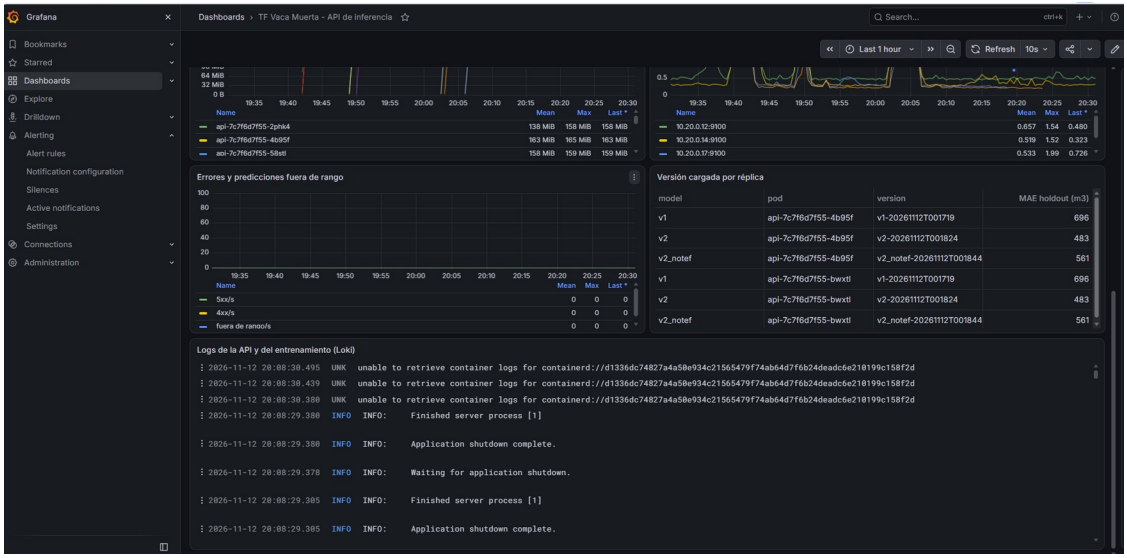

Evidencia 7.9c. RESULT sin keep-alive: 247,7 req/s, p50 255 ms, p95 1.962 ms, 0 % errores



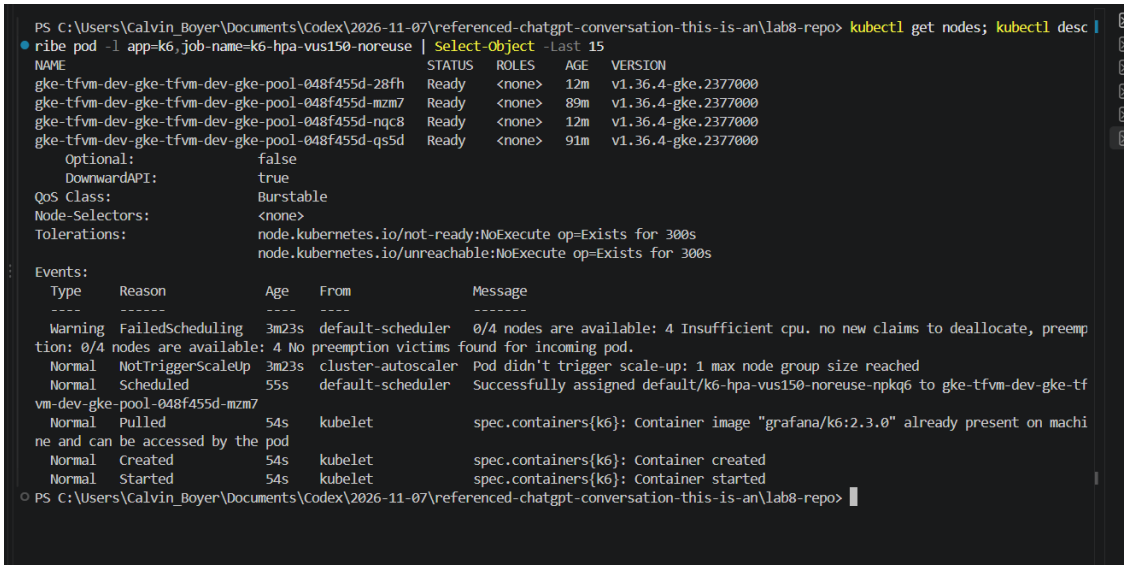
Evidencia 7.9d. Dashboard: deseadas 10, disponibles 7, pico 295 req/s, CPU repartida entre réplicas



Evidencia 7.10a. Dashboard final, última hora: las tres pruebas con HPA (≈130, ≈240 y 295 req/s; réplicas a 5, 9 y 10), CPU por pod y por nodo

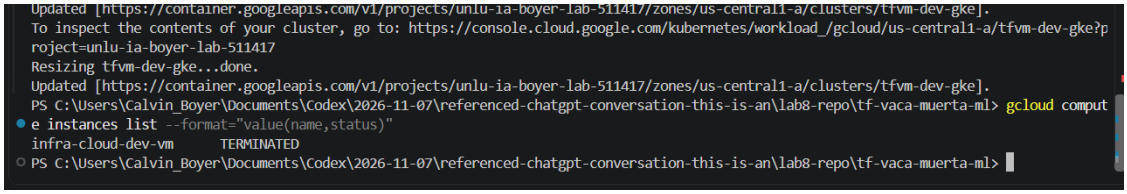


Evidencia 7.10b. Errores 5xx/4xx y fuera de rango en 0 durante todas las pruebas; versión cargada por réplica; logs en Loki

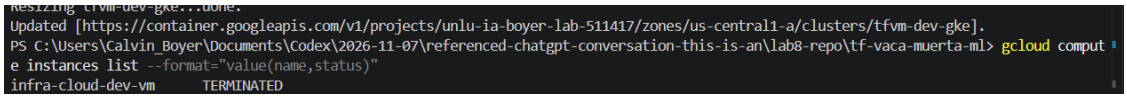


Evidencia 7.8. Pod de k6 en `Pending` 2,5 min: `FailedScheduling` *0/4 nodes are available: 4 Insufficient cpu* y `NotTriggerScaleUp` *max node group size reached* (4 nodos, el máximo); se programó cuando el HPA bajó réplicas y liberó CPU

8. Estado final (equivalente al 6.1)



Evidencia 8.1. Pausa nocturna del 11/11: sin nodos de GKE corriendo



Evidencia 8.1b. Pausa del 12/11 tras las pruebas: sin nodos de GKE corriendo